AI Agent 开发岗 求职必备

Agent 面试题库

把 Agent 聊透，面试官问不倒

100 道大厂高频真题 · 框架选型到工程落地全覆盖

100

ReAct · MCP · RAG · 多智能体 · 记忆 · 评测

目录

第零章 认知篇：Agent 面试到底考什么

0.1 面试官怎么判断你真做过 Agent

0.2 初级 / 中级 / 高级的能力分水岭

0.3 怎么用这份题库做技术自检

第一章 Agent 基础原理（10 题）

1.1 什么是 Agent，跟普通 LLM 应用差在哪

1.2 ReAct 范式：推理与行动的循环

1.3 规划与反思：Agent 怎么决定下一步、怎么纠错

1.4 工具调用 / Function Call 的本质

1.5 上下文与状态管理：长任务怎么不丢信息

1.6 确定性 vs 自主性：什么时候该收住

1.7 Agent 的失败模式（循环 / 幻觉 / 卡死）怎么排查

1.8 单 Agent 能解决的，为什么别上多智能体

1.9 Agent 的可控性怎么保证

1.10 一个最小可用 Agent 该怎么搭

第二章 框架与选型（8 题）

2.1 框架定位与适用边界

2.2 自研与开源的权衡

2.3 低代码平台与多智能体编排

2.4 记忆实现与开源模型

2.5 选型沟通与落地

第三章 RAG 与知识增强（10 题）

3.1 RAG 在 Agent 中的角色与集成

3.2 检索底座选型与索引工程

3.3 进阶检索范式

3.4 评估与知识时效性

3.5 异常与冲突处理

第四章 工具调用与 MCP（8 题）

4.1 Function Call 机制与设计

4.2 容错与权限控制

4.3 MCP 协议原理

4.4 工具集工程化与可观测性

第五章 多智能体协作（8 题）

5.1 该不该上多智能体

5.2 编排与角色分工

5.3 通信与状态

5.4 冲突与一致性

5.5 人机协同

5.6 成本、延迟与工程化

第六章 记忆与状态（7 题）

6.1 记忆的分层模型

6.2 记忆的存储与检索

6.3 上下文压缩与保真

6.4 记忆污染与遗忘治理

6.5 用户画像沉淀

6.6 长任务状态管理

第七章 工程落地与评测（11 题）

7.1 成本与延迟优化

7.2 并发控制与可观测性

7.3 评测框架与 LLM 评委

7.4 bad case 闭环与灰度发布

7.5 安全治理

7.6 评测集构建与 SLA 守护

第八章 系统设计与案例（8 题）

8.1 垂直场景 Agent 设计

8.2 Agent 与外部系统的集成边界

8.3 工程化与高可用架构

8.4 项目落地与排期

第九章 大厂面经真题复盘（30 题 · 网传整理）

9.1 字节（8 题）

9.2 腾讯（8 题）

9.3 阿里（7 题）

9.4 美团（7 题）

能力维度速查表

一、基础原理（认知 Agent 的底层逻辑）

二、框架与选型（知道该用什么、为什么）

三、RAG 与知识增强（Agent 的事实底座）

四、工具调用与 MCP（让 Agent 长出手脚）

五、多智能体协作（当单 Agent 不够时）

六、记忆与状态（Agent 的脑子）

七、工程落地与评测（从 demo 到生产）

八、系统设计与案例（综合实战）

九、大厂面经真题（按厂刷，最贴近实战）

想要这份 Agent 面试题库的配套能力速查表?

扫码添加一凡微信，备注「Agent 面试题库」



微信搜 xtyifan1204 也能找到一凡

内部资料 · 限量发放 · 严禁外传

AI Agent 开发岗面试真题库配套资源，刷完这 100 题别有收获

这份资料写给两类人：正在准备 Agent 开发岗面试的工程师，和已经在岗想补齐实战短板的。九章真题加一张能力速查表，每题都标了考察维度。求职的当备考清单刷，在岗的拿它照镜子找薄弱项。

第零章 认知篇：Agent 面试到底考什么

这一章不考具体技术，考的是你到底有没有真上手做过 Agent。很多候选人简历写满了 LangChain、RAG、多智能体，一被追问项目里最坑的那一跤就露馅。面试官判断你是不是纸上谈兵，套路其实很固定，提前知道能少踩很多雷。

0.1 面试官怎么判断你真做过 Agent

题 1：面试官靠什么判断一个候选人真的做过敏捷 Agent，而不是背了八股？

考察点：项目深挖、经验真实性

开口怎么答：

我不会等面试官来戳，我会主动把做过的项目里最踩坑的那一段讲出来。真做过的人，踩坑的细节是编不出来的，坑的深度就是你经验的深度。

回答框架：

- 面试官的判断逻辑：他不在乎你用了什么框架，在乎你能不能讲清一个需求从模糊到落地的全过程，尤其是你做过的取舍和背后的代价。
- 最有效的信号是具体数字和具体失败：这个 Agent 线上跑了多少 QPS、一次 bad case 让你查了多久、长上下文把成本顶到多少，这些细节背不出来。
- 第二个信号是你对边界的诚实：真做过的人会主动说哪些场景 Agent 不行、哪些坑劝退，而不是什么都吹能解决。
- 反信号：张口就是「我们用 LangChain 搭了一个 Agent」，往下问就空，这基本等于没做过。

面试官追问：

- 追问：那如果项目很顺利没踩坑呢？怎么接：我会讲清楚我预判了哪些坑并提前规避了，比如上线前就做了工具调用的超时降级，这比踩了坑更显经验。
- 追问：怎么证明你不是只调了 API？怎么接：我会讲我对 prompt、工具集、重试策略做过哪些 A/B，哪些改动让成功率从多少涨到多少。

避坑：

- 别把框架名当经验，面试官听到「用 LangChain 做了 Agent」后面一定追细节，接不住就塌。
- 别只讲成功不讲失败，越完美的项目越像编的，诚实讲坑反而加分。

0.2 初级 / 中级 / 高级的能力分水岭

题 2：初级、中级、高级 Agent 工程师，能力分水岭到底在哪？

考察点：能力层级、自我定位

开口怎么答：

我理解的区分不在会多少框架，在出问题时你停在哪一层。初级停在调通，中级停在能跑稳，高级停在设计上就让它能稳。

回答框架：

- 初级：能照文档把一个 Agent 跑起来，调通 Function Call、接上 RAG，但一出问题就懵，说不清为什么失败。
- 中级：能定位问题，知道长上下文怎么爆、工具超时怎么降级、幻觉怎么兜，能把一个 Agent 稳定跑在生产。
- 高级：在写第一行代码前就想清楚架构边界，知道哪些该用单 Agent、哪些该拆多智能体、成本上限怎么卡、评测怎么量化，能扛住业务从 demo 到万级 QPS 的重构。
- 面试落点：初级被问实现，中级被问排错，高级被问权衡和设计，你答的层次要匹配你投的级别。

面试官追问：

- 追问：我投中级，要不要往高级的话术靠？怎么接：别硬靠，中级里把排错和稳定性讲透就够了，硬装高级容易被追问拆穿。
- 追问：怎么在简历里体现层级？怎么接：写清楚你负责的那段里做了什么决策、带来什么量化结果，而不是列工具名。

避坑：

- 别用框架数量证明水平，面试官看的是你解决问题的深度。
- 别高估也别低估自己，层级说错会让后面所有问题都答得拧巴。

0.3 怎么用这份题库做技术自检

题 3：这 100 道题的题库，怎么用效率最高？

考察点：学习方法、自我驱动

开口怎么答：

我不建议从第一题背到最后一题。我会先拿第九章的大厂原题摸底，看自己卡在哪类问题，再倒回去补对应的章节。

回答框架：

- 摸底逻辑：大厂原题最贴近真实考察，先刷能暴露薄弱项，比顺序刷高效。
- 按轴刷：每题都标了能力维度，卡住的那根轴集中补，不要平均用力。
- 输出倒逼：每题试着自己先开口答一遍，再对照框架找缺口，比 passive 看答案记得牢。
- 复盘闭环：答不出的题标记下来，两周后重答，还卡说明没真懂。

面试官追问：

- 追问：刷题和实际做项目哪个更重要？怎么接：项目是根，题是照妖镜，没项目刷题容易飘，有项目不刷题容易讲不清。
- 追问：你平时怎么跟进行业？怎么接：我会跟几个开源 Agent 框架的 release 和论文，比如 LangGraph、AutoGPT 的更新，看工程趋势。

避坑：

- 别只背答案不练开口，面试要的是当场讲清楚，不是默写。
- 别贪多求全，100 题能讲透七成，比全背一遍但都浮在表面强。

第一章 Agent 基础原理（10 题）

1.1 什么是 Agent，跟普通 LLM 应用差在哪

题 1：到底什么是 Agent，它跟普通的 LLM 应用（比如一个问答机器人）本质区别在哪?

考察点：基础概念、认知边界

开口怎么答：

我的理解，普通 LLM 应用是「你问一句它答一句」，Agent 是「给它一个目标，它能自己想办法、调工具、看结果、再决定下一步」。区别不在模型，在闭环。

回答框架：

- 核心差异：普通应用里 LLM 只做生成，流程是人写死的；Agent 里 LLM 同时做决策，决定调哪个工具、下一步干嘛，形成一个「感知-决策-行动-观察」的循环。
- 三个要件：一个会推理的模型、一组能用的工具、一段能存状态的内存。三者齐了才叫 Agent，少一个就是普通应用。
- 自主程度是光谱：从固定流程里嵌一个 LLM 判断，到完全自治的多智能体，都是 Agent，不是非黑即白。
- 为什么火：因为 LLM 有了推理和工具，能把「想」和「做」连起来，这是以前规则系统做不到的。

面试官追问：

- 追问：那 RAG 算 Agent 吗？怎么接：不算完整 Agent，RAG 是检索增强生成，没有行动循环，但它是 Agent 常用的一个能力模块。
- 追问：Agent 和 workflow 怎么区分？怎么接：workflow 是流程写死、LLM 只填空；Agent 是 LLM 决定走哪条路，这是本质分水岭。

避坑：

- 别把「用了 LLM」就等同于 Agent，有没有自主决策循环才是关键。
- 别一上来就讲框架，先把要件和闭环讲清楚，面试官才认你懂本质。

1.2 ReAct 范式：推理与行动的循环

题 2：ReAct 范式是什么，推理和行动是怎么循环起来的?

考察点：核心范式、实现理解

开口怎么答：

ReAct 是 2022 年 Yao 等人提的范式，全名是 Reason 加 Act。它让模型交替产生「思考」和「行动」，行动调工具拿回「观察」，再用观察和思考决定下一步，直到能给出答案。

回答框架：

- 循环结构：Thought（我该怎么推进）→ Action（调哪个工具、传什么参数）→ Observation（工具返回的结果）→ 再 Thought，如此往复。
- 为什么有效：把推理过程外显成 Thought，模型能把复杂任务拆成小步，每步有工具反馈兜底，比一次性生成更准更可控。
- 实现层面：在工程里，Action 通常就是一次 Function Call，Observation 是把工具结果塞回上下文，Thought 是模型下一轮的输出。
- 工程变体：实际系统很少严格按论文跑，更多是「模型决定调工具、执行、结果回灌、模型再决策」的简化循环，还可能加停止条件和最大步数。

面试官追问：

- 追问：Thought 一定要让模型输出吗，能不能省略？怎么接：可以省，直接让模型输出 Action，但保留 Thought 能提升复杂任务的可调试性和准确率，代价是多一次生成。
- 追问：ReAct 和 Plan-and-Execute 区别？怎么接：ReAct 边想边做，Plan-and-Execute 先出完整计划再逐步执行，后者长任务更稳但灵活性差。

避坑：

- 别把 ReAct 说成某个框架的特性，它是思想范式，LangChain、原生 API 都能实现。
- 别忽略工程约束，真实系统必须加最大步数和超时，否则容易死循环。

1.3 规划与反思：Agent 怎么决定下一步、怎么纠错

题 3：Agent 怎么决定下一步做什么，又怎么发现自己走错了去纠错？

考察点：规划机制、反思能力

开口怎么答：

Agent 决定下一步，靠的是两种思路：一种是边走边看（ReAct 式），一种是先规划再执行。纠错则靠把「反思」显式成一步，让模型自己评价刚才那步走得对不对。

回答框架：

- 规划：简单任务靠 ReAct 逐步推理；复杂任务先让模型产出一份计划（子目标列表），再逐个执行，典型如 Plan-and-Execute、BabyAGI 的思路。
- 反思：Reflexion 这类工作把反思写成一段文本存进内存，模型下次决策前先读自己的反思，避免重复犯错。
- 纠错触发：常见信号包括工具返回报错、结果明显偏离目标、超过步数预算，这时模型要么重试、要么换路径、要么申请人工。
- 工程落地：我会给 Agent 一个「自检」步骤，每几步让它判断是否还朝着目标走，偏离就回滚到上一个检查点。

面试官追问：

- 追问：计划和实际脱节怎么办？怎么接：计划只是初始假设，执行中 Observation 会修正它，我会让模型在关键节点重新规划，而不是死守原计划。
- 追问：反思会不会让成本翻倍？怎么接：会的，所以反思只在长任务或失败节点触发，简单任务不反思，省成本。

避坑：

- 别把规划说成一次性产出就完事，真实系统里计划要能被 Observation 动态修正。
- 别忽视反思的成本，乱用会拖慢又烧钱，要讲清触发条件。

1.4 工具调用 / Function Call 的本质

题 4：工具调用或者说 Function Call，本质是什么？

考察点：工具机制、本质理解

开口怎么答：

本质就是给 LLM 开了一个「标准接口」，让它能用结构化参数去调外部能力，比如查数据库、发请求、算数。模型本身不出门，Function Call 是它的手。

回答框架：

- 机制：模型不直接执行代码，而是输出一段符合 JSON Schema 的结构化调用请求，由宿主程序解析、执行、再把结果塞回上下文。
- 为什么用 Schema：自然语言的参数提取太不可靠，Schema 把「调哪个函数、传什么类型参数」锁死，准确率才上得去。
- 和代码执行的区别：Function Call 是调你预定义的工具，安全可控；直接让模型写代码执行风险高，一般只在沙箱里用。
- 工程要点：调用前要做参数校验，调用后要处理超时和异常，结果太长要截断，这些都是宿主的职责不是模型的。

面试官追问：

- 追问：模型怎么知道该调哪个工具？怎么接：靠系统提示里给的工具描述和 Schema，模型按语义匹配，所以工具描述写得好不好直接决定调用准不准。
- 追问：并行调用怎么处理？怎么接：当多个工具无依赖时，我会让模型一次输出多个调用，宿主并发执行再汇总，省一轮轮等待。

避坑：

- 别把 Function Call 说成模型自己执行了函数，它只产出请求，执行在服务端。
- 别忽略参数校验，模型会乱填参数，不校验生产必出事。

1.5 上下文与状态管理：长任务怎么不丢信息

题 5：长任务里上下文怎么管理，信息才不会丢？

考察点：上下文工程、长程记忆

开口怎么答：

长任务最怕上下文窗口被塞爆或者关键信息被挤出，我的思路是「分层存、按需取」，不把所有东西都堆在 prompt 里。

回答框架：

- 分层：工作记忆放上下文窗口（当前几步），长期记忆放外部存储（向量库或数据库），用的时候再检索回来。
- 压缩：对话或任务历史过长时，先做摘要压缩再入上下文，保留决策相关的关键事实，丢掉闲聊。
- 结构化状态：用一份显式的状态对象（JSON）记录任务进度、已得结果、待办，模型每步读写它，比纯文本更稳。
- 检索回填：需要早先信息时，用向量检索或关键字从长期记忆捞回来，而不是指望窗口装得下。

面试官追问：

- 追问：压缩会不会丢关键信息？怎么接：会，所以压缩要保留决策点和未决问题，闲聊可丢，关键事实用结构化状态保底。
- 追问：上下文窗口越来越长，还需要这套吗？怎么接：长窗口缓解但没根治，成本和注意力退化还在，分层管理在长任务里依然必要。

避坑：

- 别以为窗口够大就万事大吉，信息一多模型注意力会退化，关键事实照样抓不住。
- 别把所有历史无脑塞进 prompt，成本和效果都会崩。

1.6 确定性 vs 自主性：什么时候该收住

题 6：什么时候该让 Agent 自主，什么时候得把缰绳收紧？

考察点：可控性权衡、场景判断

开口怎么答：

我的原则：风险越高、可逆性越差的动作，自主性越要收。查个资料可以放手让它跑，转一笔钱必须卡死审批。

回答框架：

- 按风险分级：读操作（查、搜、算）可以给高自主；写操作（发消息、调接口改数据）要加确认或人工；资金、权限类操作必须 HITL。
- 按可逆性：动作能回滚就敢放手，不能回滚就收紧。
- 按确定性需求：结果必须可复现的场景（如合规报表）用确定性流程，别交给随机性的 LLM 拍板。
- 工程手段：用工具权限白名单、调用次数上限、敏感操作二次确认来收缰绳，而不是靠模型自觉。

面试官追问：

- 追问：完全自主的 Agent 还有价值吗？怎么接：有，在低风险高重复场景（如自动整理文档）自主性能大幅降本，但要配好护栏。
- 追问：怎么跟业务方定边界？怎么接：把动作按风险列成矩阵，和业务、法务一起定哪类要审批，落到工具权限配置里。

避坑：

- 别一刀切全自主或全锁死，按风险分级才是工程做法。
- 别靠 prompt 提醒模型「要谨慎」，要用系统层的权限和确认硬卡。

1.7 Agent 的失败模式（循环 / 幻觉 / 卡死）怎么排查

题 7：Agent 常见的失败模式有哪些，怎么排查循环、幻觉和卡死？

考察点：故障排查、实战经验

开口怎么答：

我按现象分三类：转圈出不来（循环）、瞎编（幻觉）、卡住不动（卡死）。排查都靠埋点和回放，不是靠猜。

回答框架：

- 循环：模型重复调同一个工具或绕圈，通常是目标不清晰或没设停止条件。查法看 trace 里 Action 序列是否重复，解法是加最大步数和「已完成」判断。
- 幻觉：模型基于错误 Observation 或脑补给出假结论。查法对比工具返回和最终答案，解法是强制引用工具结果、加事实校验。
- 卡死：工具超时、返回格式错、Schema 不匹配导致解析失败。查法看错误日志里卡在哪一步，解法是超时降级和严格参数校验。
- 通用手段：每一步留 trace，出问题能完整回放，比凭记忆复盘准得多。

面试官追问：

- 追问：怎么自动发现循环？怎么接：我会对近期 Action 序列做相似度检测，连续几步高度重复就触发终止，而不是等步数上限。
- 追问：幻觉能彻底消除吗？怎么接：不能，工程上靠检索约束和事实校验把概率压低到可接受，关键场景加人工复核。

避坑：

- 别等用户投诉才发现失败，要在 trace 和评测里主动暴露。
- 别只加最大步数不治本，要定位循环根因（目标或反馈缺失）。

1.8 单 Agent 能解决的，为什么别上多智能体

题 8：单 Agent 能解决的事，为什么我反对轻易上多智能体？

考察点：架构权衡、过度设计意识

开口怎么答：

多智能体不是银弹，它带来的通信成本、调试难度、一致性问题，往往比它解决的问题还多。我的默认是单 Agent 能搞定就别拆。

回答框架：

- 成本：多智能体每轮多几次 LLM 调用和消息传递，token 和延迟都翻倍，单 Agent 一步能干的拆成三个 Agent 纯属烧钱。
- 调试：单 Agent 的失败链是线性的，多智能体的失败是网状的，一个 Agent 出错会传染一片，定位极难。
- 一致性：多个 Agent 各自维护状态，容易各说各话，要额外做对齐和仲裁。
- 何时才拆：任务真的异质（比如一个做研究一个做写作）、或单上下文装不下、或要并行提速，才考虑多智能体。

面试官追问：

- 追问：那什么场景多智能体确实更好？怎么接：强隔离的职责（研究 vs 写作 vs 评审）、超长任务需要分工、或要并行处理独立子任务时，多智能体才划算。
- 追问：怎么说服想上多智能体的同事？怎么接：让他先算账，把 token 和调试成本摆出来，多数情况单 Agent 加好工具集就够了。

避坑：

- 别把多智能体当潮流硬上，过度设计是 Agent 项目最常见的坑。
- 别用多智能体掩盖单 Agent 没调好的问题，先调好单 Agent。

1.9 Agent 的可控性怎么保证

题 9：Agent 的不可控性怎么治，怎么让它「听话」？

考察点：可控性、工程护栏

开口怎么答：

LLM 本身有随机性，可控性不能指望模型自觉，得靠系统层硬约束。我的做法是权限、格式、流程三道闸一起上。

回答框架：

- 权限闸：工具白名单加敏感操作审批，模型想越界也调不动。
- 格式闸：用 JSON Schema 和结构化输出锁死输出格式，再用校验兜底，格式乱就重生成。
- 流程闸：关键节点设人工确认和回滚点，出事能快速止损。
- 评测闸：上线前用 bad case 集压测，把不可控的场景提前暴露，而不是等线上翻车。

面试官追问：

- 追问：结构化输出失败率高怎么办？怎么接：用支持原生 structured output 的模型或后处理校验重试，失败超阈值走降级，别让格式错卡死主流程。

- 追问：随机性怎么压？怎么接：关键决策降低温度，甚至走确定性规则分支，把随机性隔离在非关键路径。

避坑：

- 别只靠 prompt 说「请严格按格式」，模型会忘，要靠 Schema 和校验硬卡。
- 别等上线才发现不可控，评测集里就要覆盖异常输入。

1.10 一个最小可用 Agent 该怎么搭

题 10：如果让你从零搭一个最小可用的 Agent，你会怎么搭？

考察点：落地能力、工程条理

开口怎么答：

我会先搭一个能跑通闭环的最小骨架，不追求花哨，核心是「模型 + 工具 + 循环 + 内存」四件套跑起来，再加护栏。

回答框架：

- 骨架：一个 LLM 调用入口，一份工具注册表（含 Schema），一个 ReAct 式循环（思考、调工具、回灌、再思考），一份内存（上下文加外部存储）。
- 第一个工具：先接一个最实用的，比如搜索或代码执行，让 Agent 立刻有「手」。
- 护栏：加最大步数、超时、参数校验，先把不死循环和不崩搞定。
- 验证：用一两个真实小任务端到端跑通，再补更多工具和评测，而不是一口气铺开。

面试官追问：

- 追问：用框架还是裸写？怎么接：demo 阶段裸写更快看清机制，真要做生产我倾向用 LangGraph 这类有状态管理的框架，省自己造轮子。
- 追问：最小 Agent 怎么验证它真的「活」了？怎么接：给它一个需要多步工具调用的任务，看它会不会自己拆步、调工具、用结果，而不是一步答完。

避坑：

- 别一上来就堆框架和十几个工具，最小骨架跑通比功能多更重要。
- 别漏护栏，没最大步数和超时，demo 都可能转圈转到你怀疑人生。

第二章 框架与选型（8 题）

2.1 框架定位与适用边界

题 1：LangChain vs LangGraph 适用边界

考察点：能否说清链式编排与图式状态机的取舍

开口怎么答：

我觉得这俩不是替代关系，是抽象层级不同。LangChain
解决的是「把模型、工具、提示词拼成一条链」的问题，LangGraph
解决的是「带状态、可循环、可人工介入的有向图」问题。我一般按业务是否要回环和分支来选。

回答框架：

- LangChain 的 LCEL 用 Runnable 组合子做声明式管道，适合线性 RAG 检索、单次工具调用这类无状态任务，链式吞吐和可读性都好。
- LangGraph 引入节点、边、状态（State）、检查点（Checkpointer），支持 cycles 和 human-in-the-loop，本质是持久化状态机，对应需要反思、重试、多轮决策的场景。
- 工程分水岭在「是否要回环」。ReAct 这类 Thought-Action-Observation 循环、自我纠错、长任务断点续跑，用 LangChain 硬写会陷入回调地狱，LangGraph 的状态图天然支持。
- 复杂度代价：LangGraph 带来图定义、状态归约（reducer）、持久化存储的学习与运维成本，小脚本上 LangChain 更轻。
- 实践里常混用，LangGraph 节点内部仍用 LCEL 链，二者同源（LangChain 公司出品），迁移成本可控。

面试官追问：

- 追问：状态怎么持久化？怎么接：用 MemorySaver 做内存态，生产换 Postgres/Redis 检查点，配合 thread_id 做会话级断点续跑，崩溃后可 resume。
- 追问：LangGraph 和直接用代码写 while 循环比优势在哪？怎么接：内置状态快照、流式事件、并发节点、可视化追踪，把「控制流」从「业务逻辑」里抽出来，便于测试和回放。

避坑：

- 别为了用图而用图，纯线性流程上 LangGraph 反而增加噪声。
- 别把全局可变状态塞进节点闭包，必须用 State reducer 显式归约，否则并发执行会丢数据。

题 2：AutoGPT / MetaGPT / CrewAI 设计哲学对比

考察点：能否识别自主智能体与角色协作范式的本质差异

开口怎么答：

这三家我更愿意按「智能体是被谁驱动」来区分。AutoGPT
是目标驱动的单智能体自我分解，MetaGPT 是流程驱动的角色软件公司，CrewAI

是任务驱动的角色协作班组。抽象层不同，落地难度也不同。

回答框架：

- AutoGPT
  走「目标递归拆解加自生成子任务」路线，把大目标塞进系统提示让模型自循环，2023 年引爆自主智能体概念，但缺乏约束容易跑飞、token 消耗失控。
- MetaGPT 把 SOP 写进代码，用 ProductManager、Architect、Engineer 等角色和消息总线模拟软件公司，强调「结构化输出加阶段门禁」，偏重可复现的软件生成流程。
- CrewAI 主打轻量多角色（Agent、Task、Crew、Process），Process 支持 sequential 和 hierarchical，和 LangChain 生态解耦，上手快、调度透明。
- 哲学差异在「控制流来源」：AutoGPT 交给模型自由发挥，MetaGPT 交给硬编码流程，CrewAI 交给显式任务图，可靠性递增、灵活性递减。
- 局限：AutoGPT 类自由度高但不可控，MetaGPT 重但偏特定领域，CrewAI 协作粒度粗、缺少内置反思回路，复杂决策仍要靠 LangGraph 补。

面试官追问：

- 追问：真实生产你会选哪个？怎么接：需求是稳定流水线选 MetaGPT 思路，快速搭多角色 demo 选 CrewAI，需要强约束长任务用 LangGraph 编排 CrewAI 的 agent。
- 追问：为什么 AutoGPT 后来退热？怎么接：缺少 halting 条件和成本护栏，任务容易无限递归，企业场景要的是可观测、可中断、可审计，自由智能体暂不达标。

避坑：

- 别把三者当竞品，它们解决的是不同抽象层的问题。
- 别迷信「全自动」，生产环境一定要加预算上限、步骤上限和人工审批节点。

2.2 自研与开源的权衡

题 3：自研 vs 开源框架的权衡

考察点：能否从团队、成本、可控性维度做工程决策

开口怎么答：

我的判准很简单，看「团队要为哪个环节付认知税」。开源框架帮你扛通用编排，自研帮你吃掉业务特有的硬骨头。早期我一定站开源，业务跑通后再局部自研。

回答框架：

- 开源收益：LangChain 生态覆盖 600 加集成、社区踩坑多，能把「接模型、接向量库、接工具」的脏活标准化，上线周期从月级压到周级。
- 自研收益：核心链路可控，没有框架抽象的隐性开销（如 LCEL 的序列化层），延迟、成本、可调试性都能压到极致，适合高频核心场景。
- 代价权衡：开源有版本碎片化、breaking change 风险（LangChain 早期 API 频繁变动），自研有维护人力、人才依赖、隐性 bug 自己扛。

- 决策矩阵：非核心、变化快、要快上线的用开源；核心、高并发、强定制的自研或 fork 改造。
- 折中路线：开源做底座，在关键节点（记忆、调度、评测）自研插件替换，避免被框架绑架也保住迭代速度。

面试官追问：

- 追问：被框架绑架怎么看出来？怎么接：当你为了适配框架而改业务语义、或者升级版本要改大量业务代码时，就是耦合过深，该抽出自研模块。
- 追问：小团队怎么选？怎么接：小团队别自研，用 LangGraph 加少量胶水代码，把精力放在业务 prompt 和评测上，框架债以后再说。

避坑：

- 别在 demo 阶段就自研，重复造轮子拖慢验证。
- 别无脑全量引入大框架，按需引子模块，否则依赖体积和升级成本都难收场。

2.3 低代码平台与多智能体编排

题 4：Dify / Coze 类低代码平台能走到哪

考察点：能否界定低代码平台的能力天花板与适用面

开口怎么答：

我把这类平台定位成「内部工具和原型的加速器」，不是生产级 Agent 的主战场。它能让不懂代码的人三天搭出可用流程，但遇上复杂状态机和私有化深度集成就会见顶。

回答框架：

- Dify 开源可私有化，提供工作流 DSL、RAG 管线、模型路由和 API 暴露，适合企业内部知识库问答、客服脚本这类标准化场景。
- Coze（字节系）强在插件市场和一键发布到多 IM，零代码编排体验好，但闭源、数据在云端，合规敏感业务受限。
- 天花板在「编排表达力」：低代码是 DAG 加节点配置，表达不了带循环、条件回溯、动态子图的复杂 Agent 逻辑，复杂决策仍要回代码。
- 工程现实：低代码生成的流程难做单测、难版本化 diff、难做灰度，团队协作和代码评审会掉队。
- 合理用法：用它们做需求验证和 MVP，验证完把核心逻辑用 LangGraph 重写进主系统，而不是长期寄生在平台上。

面试官追问：

- 追问：什么场景可以长期留在低代码？怎么接：流程稳定、变更少、业务方自己维护的轻量应用，比如固定话术的售前问答，留在平台最省心。
- 追问：私有化部署 Dify 要注意什么？怎么接：注意模型网关鉴权、知识库权限隔离、工作流版本管理，这些开源版要自己补运维。

避坑：

- 别把核心业务逻辑全部锁死在低代码平台，迁移成本会随节点数指数上升。
- 别忽视可观测性，低代码默认埋点弱，生产排障要自己接日志和 trace。

题 5：多智能体编排框架怎么选

考察点：能否按任务结构匹配编排范式

开口怎么答：

选框架前我先问自己，任务的依赖是「流水线」还是「协作网络」。线性依赖用 CrewAI 省事，有动态路由和回环用 LangGraph，要强一致流程用 MetaGPT 式硬编码。

回答框架：

- 按通信拓扑分：Centralized（有一个 orchestrator 派活，如 AutoGen 的 GroupChatManager）、Decentralized（peer to peer 协商，如 CAMEL）、Hierarchical（经理分解、员工执行，CrewAI hierarchical process）。
- 按状态管理分：无状态黑板（共享消息池）实现简单但易冲突；有状态图（LangGraph）能精确控制每个 agent 的可见状态和转移条件。
- 按可靠性需求：需要回滚、断点、人工介入选带 checkpointer 的图式框架；纯探索性头脑风暴可以用 AutoGen 自由对话。
- 论文支撑：Generative Agents（Park 2023）给出记忆、规划、反思三角，AgentVerse（2023）验证角色扮演提升推理，这些机制决定框架是否自带反思回路。
- 选型清单：看是否支持流式、是否支持 human-in-the-loop、是否可观测（LangSmith、trace）、是否易测。缺一项就权衡补自研。

面试官追问：

- 追问：多智能体一定会更强吗？怎么接：不一定，协作有通信开销和一致性损耗，简单任务单智能体加好 prompt 更稳更省，先测基线再决定要不要拆。
- 追问：agent 之间怎么传上下文不爆窗口？怎么接：用共享记忆库加摘要压缩，或者只传结构化结论而非原始对话，配合 token 预算截断。

避坑：

- 别一上来就拆十个 agent，协作成本会吃掉模型能力提升。
- 别让 agent 互相等待形成死锁，编排层要有超时和降级。

2.4 记忆实现与开源模型

题 6：Agent 记忆模块的主流实现

考察点：能否区分短期、长期、工作记忆及其工程落地

开口怎么答：

记忆我习惯拆成三层来看，上下文窗口是工作记忆，会话历史是短期，向量库加结构化存储是长期。这一层设计好坏直接决定 Agent 会不会「转头就忘」。

回答框架：

- 短期记忆：把对话轮次塞进 prompt，靠滑动窗口或摘要压缩（如 LangChain ConversationSummaryMemory）控长度，原生窗口模型（128K、200K）缓解但成本高。
- 长期记忆：向量数据库（Chroma、Milvus、pgvector）做语义检索，写入时切块加 embedding，读取时 top-k 召回，对应 RAG 范式。
- 工作记忆、Scratchpad：ReAct 的 Thought 轨迹、Reflexion 的自我反思文本，存成可回放的轨迹，下次失败能检索过往教训。
- 结构化记忆：用数据库或 JSON 存实体、偏好、计划进度（如 MemGPT 的 archival、recall、core memory 分层），支持精确查询而非纯语义。
- 演进代表：MemGPT（2023）提出操作系统式分层记忆，把主上下文当 CPU 寄存器、外部存当做磁盘，主动搬运而非被动截断，是工程上值得借鉴的思路。

面试官追问：

- 追问：记忆怎么避免污染和过时？怎么接：给记忆打时间戳和置信度，检索时按新鲜度加权，定期用模型做回忆冲突检测和清理。
- 追问：多会话共享记忆怎么做？怎么接：用 user_id 做隔离命名空间，跨会话的长期偏好进用户画像库，会话级临时记忆随会话销毁。

避坑：

- 别把所有历史无脑塞回 prompt，噪声会拖垮推理还烧钱。
- 别只依赖向量召回，结构化事实用精确查询更准，语义检索补周边。

题 7：开源模型跑 Agent 的可行性

考察点：能否从能力、成本、部署三角评估开源 Agent 底座

开口怎么答：

可行，但有前提。工具调用和长推理这类硬活，70B 以上量级的开源模型才稳，7B、13B 跑简单 Agent 能玩但生产别指望。我按「任务难度配模型尺寸」来控成本。

回答框架：

- 工具调用能力：Qwen2.5、Llama3.1、DeepSeek-V2、V3 已支持 function calling 和 JSON 模式，能驱动 ReAct 式 Agent，但小模型易漏参数、错格式。
- 推理与规划：复杂多步规划需要强推理底座，32B 以下在 HumanEval、BBH 等基准掉队明显，长链任务建议 70B 或 MoE 大模型（如 Qwen2.5-72B、DeepSeek-V3）。
- 部署成本：70B 全精度要多卡 A100、H100，量化（AWQ、GPTQ 4bit）能压到单卡 48G 跑，vLLM、TGI、SGLang 做吞吐和批处理，token 成本远低于闭源 API。
- 数据隐私：金融、医疗等合规场景，开源私有化部署是硬要求，闭源 API 直接出局。
- 工程取舍：用开源小模型跑高频简单子任务（分类、抽取），闭源大模型跑关键决策，混合部署降本提质，是当下工业界主流做法。

面试官追问：

- 追问：开源模型做 Agent 最大短板？怎么接：指令跟随和长上下文稳定性弱于 GPT、Claude，复杂反思回路易发散，要靠强约束 prompt 和校验层兜住。
- 追问：怎么验证开源模型能不能上岗？怎么接：拿真实轨迹集做离线评测（成功率、步骤效率、格式合规率），跑通再小流量灰度，别凭基准分数直接上。

避坑：

- 别用 7B 模型硬扛多工具复杂 Agent，格式错误会让整个流程崩。
- 别只看 MMLU 分数，Agent 看的是工具调用成功率和多步一致性，要自己造评测。

2.5 选型沟通与落地

题 8：怎么给技术主管讲清选型理由

考察点：能否用业务语言而非技术炫技完成向上沟通

开口怎么答：

我给主管讲选型，从不堆框架名词，先讲「我们要解决什么、卡在哪、选这个能省多少时间和钱」。技术细节留到他追问时再展开，主线永远是对齐业务目标。

回答框架：

- 先算账：把候选方案按开发周期、人力、推理成本、运维风险列成对比表，主管一眼看到 ROI 差异比听架构演讲有用。
- 对齐约束：点明硬约束（数据是否出域、延迟上限、合规要求），这些往往直接排除某些选项，沟通效率最高。
- 给渐进路径：说清先用开源框架快速验证，跑通后再把核心模块自研或换框架，让主管看到风险可控、不一步到位赌死。
- 用指标说话：把「为什么不选 A」落到可量化项，比如 A 缺 human-in-the-loop 会导致长任务不可中断、B 的私有化成本三年省 X 万，避免主观偏好。
- 带备选和止损点：每个方案标注触发切换的条件（如成本超阈值、准确率不达标），展现你考虑过失败路径，主管最吃这一套。

面试官追问：

- 追问：主管更关心技术还是商业？怎么接：多数情况先商业后技术，用成本和风险讲清楚，他自然信你的技术判断。
- 追问：两方案旗鼓相当怎么推？怎么接：选团队最熟、生态最活跃、招人最容易的那个，长期维护成本比纸面性能更关键。

避坑：

- 别用一堆框架名词炫技，主管要的是决策依据不是术语课。
- 别只给一个方案，没有备选就显得没做过功课，也少了谈判余地。

第三章 RAG 与知识增强（10 题）

3.1 RAG 在 Agent 中的角色与集成

题 1：RAG 在 Agent 里的角色（不只是问答）

考察点：理解 RAG 作为 Agent 记忆与工具之外的第三类知识供给方式的定位。

开口怎么答：

我觉得 RAG 在 Agent 里早就不是单纯做问答的检索外挂了。它其实是给 Agent 提供可信、可控、可审计的外部知识通道，让模型在不改权重的前提下拿到领域事实和实时数据，用来支撑规划、反思和工具调用这些核心环节。

回答框架：

- 角色一，事实供给与减幻觉。Agent 做决策时如果只靠参数记忆，容易在专有领域编内容。RAG 把企业文档、工单、API 文档切好喂进去，让结论有出处，可回溯 citation。
- 角色二，动态记忆与长期上下文。单靠 LLM context window 放不下长期状态，RAG 把历史轨迹、用户偏好、环境快照按向量存进 Milvus，Agent 每个 step 先检索再决策，等于外挂了一块可增长的记忆。
- 角色三，工具选择的依据。很多 Agent 在 ReAct 循环里要先判断该调哪个工具，检索回来的 API schema 或操作手册直接变成 action 的入参，RAG 在这里是工具路由的知识底座。
- 角色四，可审计与合规。金融、医疗场景要求每句结论能溯源，RAG 的 chunk id 和 source 字段天然支持人工复核和监管留痕。
- 论文与范式。Lewis 等人 2020 年 RAG 原论文把生成器接检索器，后来 Self-RAG 用反思 token 让模型自己决定要不要检索，把 RAG 从被动外挂变成 Agent 主动行为。

面试官追问：

- 追问：RAG 和让 Agent 直接调搜索 API 有什么区别？怎么接：RAG 是先把知识索引化、做语义切片和重排，结果稳定可控，适合私有知识。搜索 API 是实时广域检索，噪声大、不可控，适合开放式事实补全，两者常组合用。
- 追问：RAG 会不会拖慢 Agent 推理？怎么接：会，单轮检索加重排大概增加 100 到 300 毫秒，我的做法是把检索放进工具调用里异步化，并做意图预判，只在不确定时才触发检索。

避坑：

- 别把 RAG 当成万能去幻觉方案，检索不准反而让 Agent 更自信地犯错。
- 别把所有知识都塞向量库，高频稳定事实可以前置成 prompt 模板，减少无谓检索。

3.2 检索底座选型与索引工程

题 2：向量库选型：Milvus / Qdrant / pgvector

考察点：根据数据规模、运维成本和查询模式做向量数据库选型。

开口怎么答：

我选向量库不会一上来就追新，得看数据量和团队现状。十万级以内、已经在用 Postgres，我直接用 pgvector 最省事。上亿级、要分布式和 GPU 索引，我会选 Milvus。要轻量、单机和强过滤，Qdrant 很香。

回答框架：

- 规模维度。pgvector 适合百万级以内，单机 Postgres 足够，复用现有事务库。Milvus 2.x 支持十亿级向量，分布式架构、多副本，适合超大规模召回。Qdrant 单节点性能强，Rust 实现，千万到亿级够用。
- 索引算法。Milvus 提供 IVF_FLAT、IVF_PQ、HNSW、DiskANN，量化能压内存。Qdrant 主推 HNSW 加量化，过滤性能好。pgvector 默认 IVFFlat 和 HNSW（1.6 起支持），但要自建维护。
- 过滤与混合查询。Qdrant 的 payload 过滤和 Milvus 的标量字段过滤都支持带条件的向量检索，做 metadata 加向量联合查询很关键。pgvector 靠 SQL where 拼接，灵活但大表慢。
- 运维成本。Milvus 组件多，要 etcd、MinIO、Pulsar，部署重。Qdrant 单二进制或 Docker 即可。pgvector 零额外运维，但 Postgres 本身要养。
- 生态。Milvus 有 Attu 可视化和 pymilvus SDK，LlamaIndex、LangChain 都原生适配。Qdrant 有官方 client 和不错文档。pgvector 直接走 SQLAlchemy。

面试官追问：

- 追问：HNSW 和 IVF_PQ 怎么取舍？怎么接：HNSW 召回快、内存占用高，适合在线低延迟。IVF_PQ 用乘积量化压内存，精度略降，适合内存紧张、能接受稍慢的离线批召回。
- 追问：向量库要不要上云托管？怎么接：中小团队用 Zilliz Cloud 或 Qdrant Cloud 省运维，数据敏感度高的金融客户我倾向自建，避免出网合规风险。

避坑：

- 别在千万级数据上还硬撑 pgvector，HNSW 索引构建和内存会顶不住。
- 别忽略过滤条件下召回率崩塌，带 metadata 过滤时向量索引可能退化成全扫，要测带过滤的 recall。

题 3：Chunking 策略对检索效果的影响

考察点：掌握切片粒度、重叠、结构化切分对召回和生成质量的权衡。

开口怎么答：

切片看着简单，其实是 RAG 效果的天花板之一。我一般不做一刀切，长文档按语义块切，表格和代码按结构切，重叠留个 10% 到 15% 防截断，粒度太小噪声大、太大定位不准。

回答框架：

- 粒度影响。chunk 太小，一个知识点被拆散，检索回来语义不完整。太大，命中后塞进 prompt 噪声多、容易超出窗口，且一个 chunk 里混多个主题会稀释相关性。经验上 256 到 512 token 是文本通用甜区。
- 重叠策略。设 overlap 比如 64 token，让边界上下文连续，避免关键信息卡在切缝。但重叠太大会重复召回、拉低重排区分度。
- 结构化切分。Markdown 按标题层级、代码按函数、PDF 按表格和段落，比固定字符切好太多。LlamaIndex 的 MarkdownNodeParser、SentenceSplitter 可以按句和标题递归切。
- 嵌入模型匹配。bge-large、e5 这类模型对长上下文友好，可用较大 chunk。老模型上下文短，要切细并做 query 侧扩展。
- 元数据携带。每块带 source、page、heading 路径，下游重排和引用都用得上，切分时就把结构信息写进 metadata。

面试官追问：

- 追问：父子召回怎么用？怎么接：父块存大块保证上下文，子块做精细检索，命中后回链父块喂给模型，兼顾精度和完整，LlamaIndex 的 SentenceWindowNodeParser 就是这套。
- 追问：怎么评估切片好坏？怎么接：用人工标注 query 到 gold chunk 的命中率，看不同粒度下 nDCG@5 和 answer faithfulness 的变化曲线，选拐点。

避坑：

- 别用固定字数硬切，会把句子和表格切断，召回质量直接掉。
- 别盲目加重叠，重叠过大会让重排器给相似块打高分，挤掉真正不同的结果。

题 4：混合检索（向量 + 关键词）怎么落地

考察点：理解稠密与稀疏检索互补、融合排序的工程实现。

开口怎么答：

纯向量检索对专名、编号、报错码这类精确匹配很弱，我一定会上混合检索。向量抓语义，BM25 抓关键词，两者召回后用 RRF 或者交叉编码器融合，能明显抬召回。

回答框架：

- 互补原理。向量检索靠语义相似，对同义改写友好，但遇到 SKU 编码、error code、人名这种 token 级精确项会漏。BM25 这类稀疏检索正好补这个洞，术语召回稳。
- 稀疏端实现。用 Elasticsearch 的 BM25，或把 BM25 接入同样向量库，Milvus 的 hybrid search、Qdrant 的 full-text filter 都行，也可以用语义稀疏模型 SPLADE、BM25 加 ColBERT 做多向量。
- 融合策略。Reciprocal Rank Fusion（RRF）按排名倒数求和，实现简单、不用对齐分数分布，k 一般取 60。更进阶用 Cross-Encoder 对候选对重排，精度高但慢，只用在 top-k 候选上。
- 加权调参。可以给向量和关键词各设权重 alpha，离线用 dev 集调，不同业务 alpha 不一样，法律文档关键词权重高，闲聊语义权重高。

- 框架支持。LangChain 的 EnsembleRetriever、LlamaIndex 的 QueryFusionRetriever 都能拼多路召回，底层接 Milvus 加 Elasticsearch 双引擎。

面试官追问：

- 追问：RRF 和加权求和哪个好？怎么接：RRF 不依赖分数可比性，工程上更稳，我默认用它。加权求和要求两路分数归一化到同尺度，调不好会一边倒，适合有精标数据再上。
- 追问：ColBERT 这种迟交互要不要上？怎么接：ColBERT 用 token 级 late interaction，精度比单向量高一大截，但存储和检索成本高，只在精度优先、量级不大的场景用，量级大还是 HNSW 加 BM25。

避坑：

- 别只用向量检索就宣称做好了 RAG，专名漏召回是高频翻车点。
- 别把两路分数直接相加，分布不同会被某一路淹没，先归一或者用 RRF。

3.3 进阶检索范式

题 5：GraphRAG 解决了什么、代价是什么

考察点：理解知识图谱增强检索的适用边界与成本结构。

开口怎么答：

GraphRAG 我把它看成针对全局性、跨文档问题的补充方案。普通 RAG 答局部事实还行，一旦问整个行业的趋势、实体关系这种要靠聚合推理的问题就垮。GraphRAG 抽实体关系建图，再社区摘要，代价是建图贵、更新慢。

回答框架：

- 解决什么。微软 2024 年 GraphRAG 论文用 LLM 抽取实体、关系、声明，构建知识图，再用 Leiden 算法做社区划分，对每个社区生成摘要。查询时分 local 和 global 两类，global 查询能跨文档聚合出整体洞察，这是向量 RAG 做不到的。
- 局部与全局。local search 围绕实体走邻居子图，适合具体事实。global search 借助社区摘要做宏观归纳，适合这个领域主要风险有哪些这类问题。
- 代价一，建图成本。抽取和摘要全靠 LLM 调用，百万 token 文档建一次图可能花几十美元、跑几小时，远贵于切块嵌向量。
- 代价二，更新困难。知识变更要重抽或增量更新图，社区摘要要重算，实时性弱，不适合天天变的业务数据。
- 代价三，查询复杂。global 查询要 map-reduce 多个社区摘要，延迟和 token 消耗都高。

面试官追问：

- 追问：什么场景不值得上 GraphRAG？怎么接：FAQ、单文档问答、数据高频更新这类场景，普通向量 RAG 性价比高得多，GraphRAG 只在高价值分析、低频全局查询上划算。
- 追问：有没有轻量替代？怎么接：HippoRAG 用个性化 PageRank 做知识整合，LightRAG 简化了图构建和查询流程，比原版省，但本质仍要建图成本，得按预算权衡。

避坑：

- 别以为 GraphRAG 能取代向量 RAG，它擅长聚合不擅长精确检索，两者常搭配。
- 别低估建图费用，上线前一定先小样本测建图单价和时延，避免账单失控。

3.4 评估与知识时效性

题 6：检索结果怎么喂给 Agent 做决策

考察点：掌握检索上下文注入 Agent 决策循环的方式与格式设计。

开口怎么答：

检索结果不是丢一段文本进 prompt 就完事。我会把 chunk 连带 source、置信度、时间戳整理成结构化上下文，再让 Agent 在规划步显式引用，这样它既能用上知识，又知道这条知识的可靠程度。

回答框架：

- 格式设计。每条检索结果给原文、source、chunk_id、score、更新时间，用 XML 标签或者分隔符包起来，方便模型定位，也方便下游做引用高亮。
- 注入位置。可以塞 system prompt 做背景知识，也可以做成 tool 返回，让 Agent 在 ReAct 里按需调用。我倾向工具化，模型自己决定什么时候查，更接近 Self-RAG 的主动检索。
- 重排与截断。检索回来先过 cross-encoder 重排到 top 5 到 top 8，再按 token 预算截断，避免长上下文被噪声淹没，也控制成本。
- 置信度使用。把 score 和 source 新鲜度透传给决策，低置信结果模型应标注不确定，而不是硬编，配合 citation 让用户判断。
- 与记忆配合。RAG 结果和 Agent 的工作记忆分开存，每轮决策时合并，短期记忆用向量库按 session 检索，避免 context 越滚越大。

面试官追问：

- 追问：检索结果和工具返回怎么区分？怎么接：本质都是上下文，但检索结果标 retrieval 类型、带 source，工具返回标 tool 类型、带 schema，模型在 prompt 里看到类型标签就能区分用法。
- 追问：多轮对话里检索要不要每轮都做？怎么接：不一定，我会对 query 做去冗余和意图判断，重复问题直接命中对话记忆，只有新问题或事实可能变时才重新检索，省 token。

避坑：

- 别把低分结果也全塞进去，噪声会拉低决策质量，重排加截断是必须的。
- 别让 Agent 把检索内容当绝对真理，要透传 source 和时效，让模型保留怀疑。

题 7：RAG 评估：召回率之外看什么

考察点：建立覆盖检索、生成、端到端的多维评估体系。

开口怎么答：

只看召回率会误导人，召回高不代表答得对。我从检索质量、生成忠实度、答案有用性三层分别打指标，再用端到端任务成功率兜底，最好再上线上 A/B 和人工抽检。

回答框架：

- 检索层。除 recall@k、nDCG@k 看排序质量，还要看 precision 和 MRR，确认 top 结果真相关。RAGAS 的 context recall、context precision 直接量这一层。
- 生成忠实度。faithfulness 看答案是否被检索上下文支持，不编造，RAGAS 和 TruLens 都能自动算。answer correctness 综合事实对齐和语义相似。
- 答案质量。answer relevancy 看答非所问没有，groundedness 看每句能否回溯到 chunk。toxic、安全也要过一道。
- 端到端。用任务成功率或用户满意度这种业务指标，比单点指标更贴近价值，配合 LLM-as-judge 做大规模自动评。
- 持续评估。离线集加线上采样人工标注形成闭环，RAGAS 跑在 CI 里，每次改 chunk 策略或换模型就回归一遍。

面试官追问：

- 追问：LLM-as-judge 靠谱吗？怎么接：比规则准，但要防自评偏差和位置偏好，我用多模型交叉评加人工校准，关键场景保留人工抽检兜底。
- 追问：没有标注数据怎么起步？怎么接：先用 LLM 生成 synthetic question-answer 对做弱监督，再挑难例人工标，逐步攒出黄金集，别等完美数据才开评。

避坑：

- 别只报召回率，它高而排序差是常见的假象。
- 别用生成流畅度当效果好，流利地说错比说不出更危险，faithfulness 必须单独看。

题 8：知识库实时性怎么保证

考察点：掌握增量索引、CDC 同步与失效策略。

开口怎么答：

知识库实时性是我上线 RAG 最在意的点之一。静态文档周更就够了，但价格、库存、工单状态这种分钟级变动的数据，我必须靠 CDC 加增量索引，再配 TTL
和版本号兜底，不能让模型引用昨天的旧值。

回答框架：

- 增量索引。Milvus 支持 upsert 和 delete by id，文档变更只更新对应 chunk，不重建全库。Qdrant 用 pointwise upsert，pgvector 走 upsert on conflict。
- 数据同步。用 Debezium 这类 CDC 捕获数据库 binlog，消息进 Kafka，消费端触发重新切块嵌向量写库，端到端延迟能压到秒级。
- 失效与版本。每条 chunk 带 update_time 和 version，检索时按时间窗过滤，过期 chunk 直接降权或剔除，配合 TTL 自动过期。

- 混合来源。实时数据走工具调用直接查库，不必进向量库，RAG 只管相对稳定知识，动态事实交给实时查询，分层处理。
- 一致性校验。定期对源和索引做 checksum 比对，发现漂移就触发重建，避免悄悄陈旧。

面试官追问：

- 追问：实时数据和向量库冲突以谁为准？怎么接：以源系统为准，检索结果只作参考并带时间戳，强一致性要求的事实走实时 API 查询，不依赖索引。
- 追问：索引更新会阻塞查询吗？怎么接：Milvus 的 segment 滚动和 Qdrant 的写时复制能保证查询不阻塞，上线前压测读写并发确认无显著影响。

避坑：

- 别把高频变动数据硬塞向量库，更新成本和陈旧风险都高，实时查更稳。
- 别只看写入成功就当同步完成，要做端到端延迟监控，binlog 到索引的链路断了你未必知道。

3.5 异常与冲突处理

题 9：检索失败时的兜底策略

考察点：构建 RAG 系统的降级与容错链路。

开口怎么答：

检索失败我默认它一定会发生，所以系统要设计降级。召回空、置信度全低、向量库超时，这几类我都预案好了，要么放宽检索、要么切实时查询、要么老实说不知道，绝不能让 Agent 拿空上下文硬编。

回答框架：

- 判定失败。设阈值，如 top1 score 低于 0.3、或返回为空、或向量库 500 毫秒无响应，都算失败，用明确信号触发兜底，不靠模型主观判断。
- 降级一：放宽召回。空结果时退到 BM25 全量关键词召回，或把 top-k 从 5 扩到 20 再重排，常能救回一批。
- 降级二：实时查询。知识库没命中，转去调业务 API 或搜索，拿到实时答案再生成，适合事实性兜底。
- 降级三：诚实拒答。实在没有可靠来源，Agent 应明确说不确定并给建议路径，而不是编，保护可信度。
- 监控与回放。失败率告警，落日志存 query 和上下文，离线复盘漏召，反哺 chunk 策略和同义词表。

面试官追问：

- 追问：兜底会不会引入不相关数据？怎么接：放宽召回后必须过一道重排和最小相关阈值，低于阈值就走拒答，不把噪声当答案。
- 追问：超时怎么不影响用户体验？怎么接：给检索设短超时并异步，超时就用缓存或上一轮上下文顶上，同时后台重试，前端先给部分回答。

避坑：

- 别让失败静默，空上下文生成的答案最危险，必须显式兜底。
- 别把所有失败都导向拒答，先用放宽和实时查询救，只在真没来源时才说不知道。

题 10：多源知识冲突怎么处理

考察点：掌握冲突检测、溯源优先级与一致性决策。

开口怎么答：

多源冲突是 RAG 进生产后的硬骨头。我的思路是先让系统察觉冲突，再按来源权威性、时效性和置信度排优先级，把判断依据暴露给用户，而不是替用户悄悄选一个。

回答框架：

- 冲突检测。检索回来多条结果观点不一致时，用 LLM 或规则做声明级比对，标出相互矛盾的事实，RAGAS 的 faithfulness 也能辅助发现上下文内部不一致。
- 优先级规则。给 source 打权威分（官方文档高于论坛、新版本高于旧版），冲突时高权威、高时效的胜出，规则可配置，不同业务权重不同。
- 置信度融合。多源一致则提升置信，多源矛盾则降低并标注分歧，让 Agent 输出存在不同说法而非武断结论。
- 透传用户。把冲突点和各自来源列出来，让用户决策，医疗、法务这类高风险场景尤其要透明，不能黑盒选定。
- 版本化与治理。知识入库时带来源和时间，冲突本质是版本和口径问题，靠知识治理流程前置消解，比运行时硬解更稳。

面试官追问：

- 追问：权威分怎么定？怎么接：结合来源类型、发布时间、历史准确率人工加模型打分，存 metadata，检索时作为重排特征，定期复核校准。
- 追问：冲突检测和生成会互相拖累吗？怎么接：检测放检索后、生成前的轻量步，只对 top 候选做声明比对，开销可控，换来的一致性能显著降低错误结论。

避坑：

- 别让模型默认采信第一条结果，冲突时它会自信地选错边。
- 别把冲突悄悄抹平，用户有知情权，尤其高风险领域，透传来源才是负责任的做法。

第四章 工具调用与 MCP（8 题）

4.1 Function Call 机制与设计

题 1：Function Call 的 JSON Schema 怎么设计才稳

考察点：工具描述的结构化表达与 LLM 解析成功率

开口怎么答：

我觉得工具 Schema 稳不稳，直接决定了 Agent 能不能把活干对。我一般会把它当成给模型看的接口契约来写，而不是随便塞个字段就完事。

回答框架：

- 严格走 JSON Schema draft-07 的 subset，OpenAI 与 Anthropic 的 tool use 都只接受这个子集。required 字段必须显式列出，optional 字段给默认值说明，避免模型瞎填。
- 每个参数写清 description，描述里塞进取值范围、单位、非法示例。实测把约束写进 description 比单纯 type 限定更能压住幻觉参数。
- 用 enum 与 const 收敛离散取值，numeric 加 minimum 与 maximum，string 用 pattern 正则兜底。Anthropic 的 tool use 支持嵌套 object 与 array，复杂入参要拆成强类型子结构而非自由文本。
- 工具名保持动词开头加名词，长度别超 64 字符，避免同名重载。多工具并排时，模型靠名字和 description 做路由，命名越歧义越容易挑错工具。
- 对模型不可判定的字段（如分页 token）设为程序侧注入，不暴露给模型，降低误填概率。

面试官追问：

- 追问：模型经常传错类型怎么办？怎么接：在调用前加一层 schema 校验，用 pydantic 或 ajv 做 runtime 校验，校验失败直接返回结构化 error 让模型重试，而不是崩掉整条链路。
- 追问：工具太多模型选不准？怎么接：做工具检索或分组路由，按意图先筛出 top-k 候选再给模型，控制单次可见工具数在 20 个以内，实测超过 30 个准确率明显掉。

避坑：

- 把整段业务说明堆进 description，导致模型抓不住重点。
- 用 free-form string 承接结构化入参，把解析风险全甩给下游。

题 2：并行工具调用的并发与依赖处理

考察点：多工具调度的依赖图与并发控制

开口怎么答：

我碰到过一次要同时查天气、查汇率、查库存的场景，串行太慢，并行又怕有依赖。我的思路是先建依赖图，再决定哪些能一把发。

回答框架：

- 让模型一次返回多个 tool_call，Anthropic 与 OpenAI 都支持单轮多调用。框架侧按 tool_call 列表并行 dispatch，用 asyncio.gather 或线程池控制并发度，典型并发上限设 8 到 16。
- 依赖关系交给模型在 thinking 阶段显式声明，或者由规划器拆成 DAG。无依赖节点并行，有依赖节点等上游 result 回写后再发，避免用 placeholder 硬塞。
- 并发要配超时与隔离，单工具超时设 5 到 10 秒，整体回合设熔断。用 semaphore 限制对同一个外部 API 的并发，防止触发 ratelimit 把账号打挂。
- 结果回传顺序要与 tool_call_id 一一对应，OpenAI 要求每个 tool 消息带 tool_call_id，错配会导致上下文错位。
- 对幂等工具才并行，写操作（下单、扣款）必须串行并加锁，防止竞态产生重复副作用。

面试官追问：

- 追问：模型并行调用里混进了有依赖的两个工具？怎么接：在 executor 层做静态依赖检查，发现未满足前置参数的调用先挂起，回写上游结果后再补发，必要时让模型重新规划。
- 追问：并发下如何保证上下文不乱？怎么接：维护 tool_call_id 到结果的映射表，组装 messages 时严格按 id 配对，并保留原始调用顺序做日志追踪。

避坑：

- 把所有工具无脑并行，把带副作用的写操作也并发出去。
- 忽略 tool_call_id 配对，导致模型拿到张冠李戴的结果。

4.2 容错与权限控制

题 3：工具调用失败的重试与降级

考察点：工具层的容错策略与优雅降级

开口怎么答：

工具调用是 Agent 里最容易出幺蛾子的地方，网络抖动、超时、返回脏数据都常见。我的原则是可重试的错自动重试，不可重试的错就地降级。

回答框架：

- 区分错误类型：网络超时、429 ratelimit、5xx 属于可重试，用指数退避加抖动，重试 3 次，基数 200 毫秒，最大退避 5 秒。4xx 参数错误不重试，直接返给模型修正。
- 429 要读 Retry-After 头做尊重式退避，而不是盲目猛冲，否则会被长封。
- 降级策略分层：主工具挂了切备用数据源，备用也挂返回结构化兜底值（空列表或缓存快照），并在 tool result 里标注 degraded，让模型知道数据不可靠。
- 每次失败把错误摘要回写进 tool message，让模型自己决定换工具还是换说法，不要把异常堆栈直接丢给模型。
- 设 circuit breaker，某工具连续失败超阈值（如 5 次）直接短路，避免雪崩拖垮整条链路。

面试官追问：

- 追问：重试把下游打挂了怎么办？怎么接：用令牌桶限流加熔断，失败率超 50% 触发 open 状态，半开探测恢复，保护脆弱下游。
- 追问：降级返回假数据会不会误导模型？怎么接：降级值在 result 里带 explicit 标记和置信度，模型读到 degraded 字段会自动调低这部分信息的权重，必要时向用户说明。

避坑：

- 对 4xx 也做重试，纯属浪费配额还放大错误。
- 把完整 stacktrace 塞进 tool result，模型会被噪声带偏。

题 6：工具权限与安全边界

考察点：工具调用的授权模型与风险隔离

开口怎么答：

我把工具分成只读、写入、高危三类，权限必须按最小可用原则卡死。Agent 能删库这种事，我在设计上就不该让它发生。

回答框架：

- 建立能力分级：read_only 工具默认放行，write 工具需用户确认或 scoped token，destructive 工具（删除、退款、发送）强制二次确认或人工审批。
- 用 OAuth scope 或 RBAC 给每个工具绑权限点，运行时做 policy 校验，OpenAI 的 tool 也可配合 action 白名单。Anthropic 侧可在 executor 拦截未授权调用。
- 工具执行走 sandbox，文件系统、网络、环境变量用容器或 WASM 隔离，禁止工具越权读宿主敏感目录。
- 对外部工具入参做注入防御，工具描述与参数可能成为 prompt injection 载体，要对回传内容标记 trusted 与 untrusted 来源，模型侧对 untrusted 工具输出降权处理。
- 全量审计，每次工具调用记 actor、tool、参数摘要、结果状态，便于事后溯源与回滚。

面试官追问：

- 追问：用户让 Agent 执行删除但没确认？怎么接：destructive 工具在 executor 层拦一道，缺少 confirm token 一律拒绝，并把拒绝原因返给模型让它向用户要授权。
- 追问：工具被 prompt injection 诱导怎么办？怎么接：把工具输出与用户指令隔离在两个上下文通道，对工具返回的指令性文本做剥离，只允许结构化数据回流，不执行其中的命令语义。

避坑：

- 给 Agent 一把万能 token，所有工具共用一个高权限凭据。
- 信任工具回传的一切文本，把第三方内容当系统指令执行。

4.3 MCP 协议原理

题 4：MCP 协议解决了什么、和 Function Call 区别

考察点：MCP 的架构定位与协议层差异

开口怎么答：

Function Call 解决的是模型怎么表达要调工具，MCP 解决的是工具怎么被统一接进来。我理解它们是互补的，MCP 站的位置比 Function Call 更靠基础设施。

回答框架：

- MCP（Model Context Protocol）由 Anthropic 在 2024 年底提出，基于 JSON-RPC 2.0，用一套标准协议把工具、数据、上下文从各种源里解耦出来，目标是让任意模型接任意 MCP Server。
- 三种 transport：stdio 跑本地子进程，适合本机工具；Streamable HTTP 与 SSE 支持远程服务，带 session 与鉴权。这比 Function Call 纯靠 API 参数传递更工程化。
- 三类原语：Tools（模型可调用，带 JSON Schema 入参）、Resources（只读上下文，如文件或 DB 行）、Prompts（预置提示模板）。Function Call 只有工具调用这一层，没有资源与提示的概念。
- 生命周期明确：initialize 握手协商协议版本与能力，tools/list 拉清单，tools/call 执行，notifications 做被动推送。MCP SDK（TypeScript、Python、Java、Kotlin）都按这套实现。
- 与 Function Call 的关系：MCP 是传输与发现层，模型侧的 tool use 仍是 Function Call 形态。MCP Server 把工具暴露成标准接口，宿主再把它们映射成模型的 tool 定义，两者不是替代而是上下层。

面试官追问：

- 追问：有了 Function Call 为什么还要 MCP？怎么接：Function Call 每个应用要自己写工具适配，N 个模型乘 M 个工具就是 N×M 集成成本。MCP 把适配下沉到 Server 一次写好，模型侧只认协议，成本降到 N+M。
- 追问：MCP 的 JSON-RPC 和直接用 REST 有啥不同？怎么接：JSON-RPC 统一了请求响应加通知的双向通道，配合 MCP 的生命周期方法，天然支持能力协商与流式通知，REST 得自己拼一套约定。

避坑：

- 把 MCP 当成替代 Function Call 的模型能力，其实它管的是接入侧。
- 只理解 tool，忽略 resource 与 prompt 这两个常被低估的原语。

4.4 工具集工程化与可观测性

题 5：怎么给 Agent 设计一套工具集

考察点：工具集的抽象粒度与组织方法

开口怎么答：

我不主张工具越多越好，工具集设计的核心是粒度与语义清晰。我一般会按业务域切，再控制每个 Agent 可见的工具面。

回答框架：

- 按领域分组，比如检索、计算、动作、记忆四类，每组一个 MCP Server 或一个工具包，避免把所有能力塞进一个扁平列表。
- 粒度拿捏：太粗的工具内部逻辑黑盒难调试，太细的工具让模型编排成本爆炸。经验上单个工具完成一个可被命名的动作，入参不超过 6 个。
- 用 LangChain 的 @tool 装饰器或 MCP tool 定义统一描述，description 写清何时用、返回什么，让模型选得准。
- 做工具检索与动态装载，工具总量大时用 embedding 召回相关子集注入上下文，控制单次可见数，缓解选择困难。
- 留好版本与弃用通道，工具签名变更走 semver，旧版并行保留一个周期，避免线上 Agent 集体失效。

面试官追问：

- 追问：工具命名冲突或语义重叠怎么处理？怎么接：建立工具注册中心做唯一名与语义查重，重叠工具用更细的 description 区分适用边界，必要时合并。
- 追问：怎么验证工具集够用？怎么接：跑离线 eval，用真实任务集测工具选择准确率与端到端成功率，低于阈值的工具重写描述或拆合。

避坑：

- 工具堆到上百个还全量塞给模型，选择准确率直线跳水。
- 工具描述只写功能不写适用场景，模型挑错工具概率高。

题 7：长链路工具调用的可观测性

考察点：多步调用的链路追踪与诊断

开口怎么答：

长链路最怕黑盒，一步卡住全链路失联。我的做法是每一步都带 trace，把工具调用当成微服务去观测。

回答框架：

- 用 OpenTelemetry 把每次 tool_call 打成 span，串到统一 trace_id 下，记录入参摘要、耗时、状态码、token 消耗，跨 MCP Server 也透传 context。
- 在 messages 流之外单独落一份结构化调用日志，含 tool_call_id、tool 名、开始结束时间、retry 次数、降级标记，方便回放整条决策链。
- 关键指标上监控：P50 与 P95 工具延迟、失败率、重试率、降级率，配告警。Anthropic、OpenAI 也回传 usage，按步聚合算成本。
- 对工具输出做抽样留存，长文本结果存对象存储只留引用，既可控观测成本又能事后复盘。

- 提供 replay 能力，用存下的 messages 加 tool result 重跑，定位是哪一步把链路带偏。

面试官追问：

- 追问：工具多了 trace 太杂怎么看？怎么接：按业务动作聚合 span，用 trace 拓扑图折叠同类型调用，只高亮失败与慢点，避免被海量正常 span 淹没。
- 追问：敏感参数不能全量记怎么办？怎么接：日志做字段级脱敏，对 token、密码、PII 走 masking，只留类型与长度特征供排障。

避坑：

- 只记成功不记失败细节，出问题全靠猜。
- 把超长工具结果原样写日志，存储与隐私双双爆雷。

题 8：工具结果过大怎么截断 / 压缩

考察点：大结果的处理与上下文成本控制

开口怎么答：

工具返回几千行日志或长文档是常态，全塞进上下文模型既贵又容易丢重点。我的策略是近端截断加结构化压缩，必要时让模型主动再取。

回答框架：

- 设硬上限，单条 tool result 超过比如 8000 字符就截断，尾部附 [truncated, N chars omitted]，明确告知模型内容被裁，避免它误以为看全了。
- 结构化压缩优先于盲截：对列表类结果做摘要聚合，返回 count 加 top-k 代表项；对文档类先抽关键段落再用。MCP 的 resource 也建议分页读取而非一次拉全。
- 用分页与游标，tools/call 支持返回 next_cursor，模型按需翻页，避免一次性搬运大结果。
- 长结果落外部存储，tool result 只回 URL 或引用 ID，模型需要细节时再发二次调用拉取指定片段。
- 在 prompt 层约束输出粒度，让模型先声明要哪些字段，工具侧按字段投影返回，从源头削减体量。

面试官追问：

- 追问：截断会不会让模型漏掉关键信息？怎么接：截断前按相关性或时间排序保留高价值段，并在 result 里给二次检索的钩子，模型可定向回查被裁部分。
- 追问：压缩摘要会不会引入幻觉？怎么接：压缩走确定性聚合（计数、截取、关键词提取），不交给模型生成，保证摘要内容忠实于原数据。

避坑：

- 无脑把整个大结果丢进上下文，token 成本与噪声双高。
- 截断后不标注遗漏量，模型以为掌握了全部信息。

第五章 多智能体协作（8 题）

5.1 该不该上多智能体

题 1：什么时候该上多智能体，什么时候是过度设计

考察点：架构决策的边界意识，能从任务可分解性、上下文隔离、并行收益三个维度判断是否值得拆。

开口怎么答：

我判断要不要上多智能体，先看任务能不能拆成几个互不相干的子目标。如果单 Agent 加几个工具就能跑通，我绝不会为了显得高级硬拆，那只会把延迟和调试成本凭空拉高。我更在意的是，多出来的那层协作到底换来了什么，换不来就别上。

回答框架：

- 任务可分解性是前提。当子任务之间依赖弱、能并行（例如检索、抽取、校验三件事互不读对方草稿），多智能体才有正收益。AutoGen 的 GroupChat 在强依赖链场景下反而比单 Agent 慢，因为每轮都要重新 broadcast。
- 上下文隔离是核心价值。CrewAI 给每个 Agent 独立 context window，避免长任务里不同子目标互相污染 prompt。单 Agent 把代码生成和代码审查放一个窗口，审查阶段会被生成的废稿占满注意力。
- 专业角色需要不同 system prompt 与工具集。MetaGPT 用产品经理、架构师、工程师的分工把 SOP 固化进 prompt，比让一个 Agent 兼顾所有角色更稳。
- 过度设计的信号：子任务只有一两步、彼此强串行、没有明显专业分工。这时加 Orchestrator 只是把一次 LLM 调用拆成三次，token 翻倍、确定性下降。
- 我常用的落点：先单 Agent 打通最小可用，等某个环节反复出错或上下文爆掉，再针对性拆成双 Agent，这是渐进策略而不是一上来铺满。

面试官追问：

- 追问：那有没有反例，单 Agent 看着能跑但其实该拆？怎么接：我会看上下文窗口利用率，如果单一窗口里 70% 以上是历史噪声、真正有用的指令被挤到边缘，就该把噪声源拆出去单独成 Agent，保住主 Agent 的注意力预算。
- 追问：你怎么向老板证明多智能体不是过度设计？怎么接：我拿压测数据说话，对比单 Agent 和双 Agent 在准确率、P95 延迟、单次成本三个指标上的差异，只有准确率有显著提升且成本可接受才扩。

避坑：

- 别把多智能体当成万能药，所有任务都套 Orchestrator，那是把架构复杂度当技术深度。

- 别忽略串行依赖，强耦合的子任务拆开只会让 Orchestrator 反复同步状态，延迟比单 Agent 还高。

5.2 编排与角色分工

题 2：Orchestrator-Worker 模式怎么落地

考察点：掌握中心调度模式的实现机制，能说清任务分发、结果汇聚、失败重试的工程细节。

开口怎么答：

我落地 Orchestrator-Worker 时，核心是让 Orchestrator 只做调度不做重活。它负责把任务拆成工单、派给 Worker、收结果、决定能不能汇总结案，真正耗 token 的推理全在 Worker 里。这样 Orchestrator 的 prompt 短、可控，不容易被长上下文带偏。

回答框架：

- 工单结构要显式。我用 JSON schema 定义 task_id、input、dependency、expected_output，Worker 产出也按 schema 回填。LangGraph 里用 StateGraph 把每个 Worker 建成 node，边由 Orchestrator 的 router 函数按条件跳转。
- 依赖编排用 DAG 而不是扁平广播。拓扑排序后只把就绪节点入队，等上游 task_id 返回再触发下游，避免 GroupChat 那种每轮全员聊天的浪费。
- 失败重试与超时隔离。每个 Worker 配独立 timeout（我一般设 30 到 60 秒）和最多两次重试，重试仍失败就降级返回 partial result 并打标，别让一个 Worker 拖垮整条链。
- 结果汇聚阶段做一致性校验。Orchestrator 拿到多个 Worker 输出后跑一遍 schema 校验和冲突检测，比如两个 Worker 对同一字段给出不同值，就进仲裁逻辑而不是盲目拼接。
- 状态持久化。用 Redis 或数据库存 task 状态机，Worker 崩溃重启能从 checkpoint 续跑，这是生产级和玩具版的分水岭。

面试官追问：

- 追问：Worker 之间能不能直接通信，还是必须经过 Orchestrator？怎么接：我默认都过 Orchestrator 以保证可观测，但高频小消息（如中间向量）可以走共享状态层直连，前提是把那块通信单独打日志，否则出 bug 根本查不到。
- 追问：Orchestrator 自己出错怎么办？怎么接：Orchestrator 要做得极简且幂等，调度决策可重放，配合外层 supervisor 心跳检测，挂了由 supervisor 按持久化状态重建调度而非从头跑。

避坑：

- 别让 Orchestrator 越权干 Worker 的活，它一旦开始长上下文推理，调度健壮性就塌了。
- 别漏掉工单 schema 校验，Worker 输出格式漂移是多智能体出错最常见的源头。

题 4：角色分工与职责边界怎么定

考察点：能把业务目标翻译成清晰的角色契约，明确输入输出边界、工具权限和交接协议。

开口怎么答：

我定角色边界的第一原则，是每个 Agent
只对自己的输出契约负责，不替别人做决定。边界画清楚，系统才不会两个 Agent
抢同一件事或者都以为对方做了。我把角色当成接口而不是人设，这样才好测。

回答框架：

- 用职责契约而非人设描述。MetaGPT 的 Role 类把 name、profile、goal、constraints、action 全结构化，我借鉴它给每个 Agent 写一份 input/output 规范，明确能调哪些工具、不能碰哪些数据。
- 工具权限按最小原则分配。检索 Agent 只给 search 和 vector query，写代码 Agent 才有 file write 和 shell，避免越权的 Agent 误改共享状态。
- 交接协议要显式。CrewAI 的 Task 里指定 expected_output 和 context 来源，上一个 Agent 的产物作为下一个的 context 传入，边界靠 schema 卡死而不是靠 prompt 自觉。
- 重叠区用仲裁角色兜底。两个 Agent 职责相邻时，我加一个 lightweight 的 reviewer 或让 Orchestrator 做冲突裁决，不让边界模糊地带静默失效。
- 边界要可回归测试。每个 Agent 我用一组固定输入跑单元测试，断言输出符合契约，边界变动能立刻暴露，而不是等线上协作崩了才发现。

面试官追问：

- 追问：角色定太细会不会反而协作成本高？怎么接：会，角色碎到每个只干半步就成了微服务噩梦。我控制在每个 Agent 能独立完成一个有业务意义的子目标，比如「生成 SQL」而不是「写 SELECT 子句」。
- 追问：怎么处理职责边界随需求漂移？怎么接：我把角色契约放进配置而不是硬编码，需求变了只改契约和对应测试，Orchestrator 逻辑不动，这样演进成本低。

避坑：

- 别用模糊人设代替契约，写「你是资深分析师」解决不了谁负责校验的问题。
- 别让两个 Agent 共享写同一份状态又不加锁，边界再好看也会在数据层炸。

5.3 通信与状态

题 3：多智能体通信：消息 / 共享状态 / 黑板

考察点：能对比三种通信范式的适用场景、吞吐与一致性代价，给出选型依据。

开口怎么答：

我把多智能体通信分成点对点消息、共享内存、黑板三层看。选哪种不靠喜好，看 Agent 之间是强耦合对话还是弱耦合数据交换。我项目里经常是三者混用，消息管控制流、共享状态管数据流、黑板管全局认知。

回答框架：

- 消息传递（点对点）。AutoGen 的 AgentChat 和 CrewAI 的 Kickoff 走的是显式消息，Agent 之间靠 send/reply 传递，好处是因果链清晰、易回放，代价是高频率小数据交换时开销大。
- 共享状态（共享内存）。多个 Agent 读写同一份状态对象，LangGraph 的 State 就是 reducer 合并的共享字典。适合需要持续累积中间结果的场景，但要注意并发写要用 reducer 或锁，否则后写覆盖前写。
- 黑板模式（Blackboard）。Generative Agents 用全局 memory stream 当黑板，所有 Agent 往上面贴 observation、别的 Agent 按需检索。适合松耦合、角色多的系统，缺点是黑板污染和检索噪声，需要 salience 打分过滤。
- 选型依据。强顺序、需审计的用消息；需低延迟共享大块中间数据的用共享状态；角色多、交互偶然的用黑板。生产里我常把消息做控制流、共享状态做高频数据、黑板做长期记忆。
- 一致性代价。消息最强一致但最慢，共享状态要自己处理写冲突，黑板最弱一致但最灵活。我按数据重要性分级，关键字段走消息确认，非关键走黑板最终一致。

面试官追问：

- 追问：黑板模式会不会让 Agent 读到过时信息？怎么接：会，我给每条黑板记录打 timestamp 和 version，Agent 读取时按任务时间窗过滤，并且对关键决策做乐观锁，读到旧版本就重试。
- 追问：共享状态 reducer 冲突怎么解？怎么接：reducer 设计成可交换的合并（如 append 列表、取最新标量），不可交换的字段强制串行写，从机制上消灭冲突而不是靠重试兜底。

避坑：

- 别所有通信都走消息，高频共享向量用消息序列化会把延迟和 token 一起拉爆。
- 别把黑板当成万能存储，不加分心过滤，Agent 注意力会被无关 observation 稀释。

5.4 冲突与一致性

题 5：多智能体间的冲突与一致性

考察点：识别冲突来源，掌握仲裁、投票、共识机制，理解一致性等级与代价。

开口怎么答：

多智能体冲突我见得最多的是目标冲突和数据冲突两类。一个 Agent 想快、一个想准，或者两个 Agent 对同一事实给不同版本，系统不仲裁就会产出精神分裂的结果。我一般在架构层就埋好仲裁点，不让冲突静默扩散。

回答框架：

- 冲突分两类。目标冲突（角色目标互相拉扯，如生成要创意、审查要保守）靠把目标写成可量化约束并交给 Orchestrator 加权；数据冲突（同一字段多版本）靠版本号和仲裁函数。
- 仲裁机制三种。投票（多数决，适合分类类无强对的场景）、加权（按角色置信度加权，CrewAI 的 manager 可设 agent 权重）、LLM 仲裁（把分歧交给一个 judge Agent

出裁定，MetaGPT 的 Reviewer 即此思路）。

- 一致性等级要分级。强一致走消息确认和锁，最终一致走黑板异步同步。我对钱、权限类强一致，对摘要、建议类最终一致，不一刀切。
- 用共识协议约束发散。Generative Agents 里 Agent 做计划时会反思和互相影响，我借鉴其 reflection 机制，让冲突双方在汇聚前各自生成理由，再交给 judge 比证据而非比口条。
- 可观测是兜底。所有冲突和仲裁决定都落日志，含双方输入、仲裁依据、最终值，出问题能复盘是哪条规则偏了，而不是黑盒里乱判。

面试官追问：

- 追问：投票出现平票怎么办？怎么接：我预设 tie-breaker，要么交 LLM judge 按 evidence 裁定，要么按角色优先级（如安全 Agent 优先级高于效率 Agent）破平，绝不随机。
- 追问：LLM 仲裁本身不靠谱怎么防？怎么接：judge 只做结构化裁定，输入限定双方证据摘要而非全文，并要求输出理由字段，理由缺失或自相矛盾就降级到人工或上一轮结果，不盲信。

避坑：

- 别让冲突在状态层静默覆盖，后写覆盖前写是最隐蔽的 bug。
- 别把所有分歧都交给 LLM 仲裁，无明确证据的分歧让它判只会放大幻觉。

5.5 人机协同

题 6：人类在回路（HITL）怎么嵌

考察点：掌握 HITL 的触发条件、暂停恢复机制、人在回路中的决策权设计。

开口怎么答：

我嵌 HITL 的原则是只在高风险或高不确定处拦人，不是处处打断。人在回路的成本很高，每次暂停都打断自动化节奏，所以触发点要精准，比如涉及外部写操作、法律合规、或置信度低于阈值时才叫人。

回答框架：

- 触发条件显式化。我设规则触发（如调用 refund、send_email 等危险工具前必须审批）和模型触发（置信度分数低于 0.6 或自我怀疑标记时升级），LangGraph 用 interrupt_before 在指定 node 前挂起。
- 暂停恢复要无损。系统把当前 state、pending action、上下文快照全持久化，人类批准后从 checkpoint resume，不重跑前面步骤。LangGraph 的 checkpointer 配合 interrupt 天然支持这一点。
- 给人类可执行的有限选项。我不在 UI 上甩一段长文让人判断，而是给出「批准 / 拒绝 / 修改参数」三个明确动作和一句话风险说明，降低决策负担也减少误操作。
- 人在回路也要留痕。每次人工决策连同原因写入审计日志，既满足合规，也用于事后回放优化触发阈值，让 HITL 越用越准。
- 异步 human 处理。生产里人不会秒回，我把 HITL 设计成任务挂起进队列，超时未决有 fallback（如默认拒绝危险操作），不让一条等待卡死整条流水线。

面试官追问：

- 追问：人类反馈怎么反哺 Agent？怎么接：我把人工修正存成反馈样本，定期微调小模型或更新 prompt 的 few-shot，同时调触发阈值，让同类低置信情况下次更早升级或更早自动通过。
- 追问：HITL 拖慢了体验怎么平衡？怎么接：我把低风险动作默认自动放行，只拦真正不可逆的，并用批审模式把多个待决项合并给人一次处理，把人的时间成本压到最低。

避坑：

- 别把 HITL 当万能保险，处处暂停等于没自动化，人会被烦到直接全点批准。
- 别漏掉恢复无损设计，状态没存好一恢复就重跑，成本和一致性都崩。

5.6 成本、延迟与工程化

题 7：多智能体的成本与延迟怎么控

考察点：从模型分级、并行、缓存、提前退出等角度给出可量化的降本提速手段。

开口怎么答：

多智能体最容易被老板质疑的就是账单和响应时间。我的打法是模型分级加上并行和缓存三件套，大模型只用在真正需要推理的节点，能并行的不串行，能复用的一律不重算。控制得住成本，多智能体才活得到生产。

回答框架：

- 模型分级调度。Orchestrator 和简单路由用 GPT-4o-mini 或 Claude Haiku 这类小模型，只有需要深度推理的 Worker（如代码生成、复杂规划）才上调大模型。MetaGPT 里不同角色配不同模型正是这个思路，成本能差一个数量级。
- 并行压延迟。DAG 里无依赖的 Worker 用 asyncio 或线程池并发，三件独立子任务从串行 3T 降到约 T 加调度开销。AutoGen 的 GroupChat 若无谓全员等待就改成定向点对点。
- 缓存中间产物。检索结果、抽取结果按输入 hash 缓存，重复子任务直接命中。我把共享状态层和语义缓存（如 GPTCache）结合，相同查询不再进 LLM。
- 提前退出与早停。低置信或重复结论的 Worker 直接返回 cached 或 skip，不强行跑满所有 Agent。Generative Agents 的 salience 机制类似，低相关记忆不进入推理。
- 量化监控。我给每个 Agent 打 token 和耗时标签，按 node 维度看哪个最贵，针对性优化而非整体焦虑。P95 延迟超阈值就查那个最慢 node 是不是该降级模型或加缓存。

面试官追问：

- 追问：模型分级会不会让小模型误判路由？怎么接：路由逻辑我做得很轻且可测，输入是结构化工单而非自由文本，小模型失手概率低，且下游有 schema 校验兜底，错了也能被仲裁层抓到。

- 追问：缓存导致 stale 数据怎么处理？怎么接：我给缓存加 TTL 和版本号，知识类短 TTL、规则类长 TTL，关键写操作后主动失效相关缓存，不让旧结果误导后续 Agent。

避坑：

- 别为了省成本全用小模型，关键推理节点降级会直接掉准确率，省的钱不够赔错。
- 别只看平均延迟，P95 往往卡在某一两个慢 Worker，要按 node 拆开看。

题 8：多智能体调试为什么更难

考察点：理解多智能体调试的额外维度（非确定性、状态扩散、归因困难）及应对工具。

开口怎么答：

单 Agent 调试顶多追一条调用链，多智能体是 N 条链加它们之间的交互，任何一个 Agent 的随机性和状态变化都可能让结果漂移。我刚带团队做这块时，最头疼的就是「昨天还好好的今天又错了」，根因根本不在代码而在某次协作顺序。

回答框架：

- 非确定性放大。每个 Agent 调用 LLM 都有采样随机性，多 Agent 串联后误差和不确定性会累积，同样的输入跑两次结果不同，复现就比单 Agent 难一个量级。
- 状态扩散难归因。错误可能来自上游 Agent 的脏输出，经共享状态或黑板传给下游，最终在第三个 Agent 爆出来，堆栈看不到完整因果。我用全链路 trace（每个消息带 trace_id 和 parent_id）重建交互图。
- 可观测性基建是必须。LangSmith、Phoenix 这类工具能可视化 Agent 间的消息流和 token 消耗，Arize 做 LLM 评估。没有这些，多智能体调试基本靠猜。
- 确定性回放。我把每轮的 prompt、model、temperature、输入输出全落盘，支持按 seed 和快照重跑，把概率问题转成可复现问题，这是定位偶发 bug 的关键。
- 单 Agent 隔离测试打底。每个 Worker 先独立跑固定 case 证明自己没错，再上联调。联调出问题大概率是交互层而非单体，分层定位效率最高。

面试官追问：

- 追问：偶然失败怎么稳定复现？怎么接：我记录失败那次的完整快照，用相同 seed 和缓存的输入重放，并把 temperature 在调试期临时设 0，把随机性从方程里拿掉，定位到逻辑问题再放回去。
- 追问：有没有办法减少非确定性带来的调试负担？怎么接：对关键路径加结构化输出和校验，把 LLM 的自由度限制在 schema 内，不确定性从「说错话」降级为「字段不符」，前者难查后者一秒报错。

避坑：

- 别用 print 大法调试多智能体，N 个 Agent 的日志混一起只会更乱，必须上 trace 工具。
- 别在联调阶段才想起隔离测试，单体都没验证就拼装，出错根本分不清是谁的锅。

第六章 记忆与状态（7 题）

6.1 记忆的分层模型

题 1：短期 / 工作 / 长期记忆的区分

考察点：能否讲清 Agent 记忆的分层模型与各自取舍

开口怎么答：

我习惯把 Agent 的记忆拆成三层来看，上下文窗口是短期，运行态是工作，落库的是长期，这三层成本和延迟差得很远，设计时要按访问频率分层。

回答框架：

- 短期记忆就是 LLM 上下文窗口里的内容，受窗口上限约束，比如 128K token，零检索延迟但最贵，进程掉线即失
- 工作记忆类比 MemGPT 的 main context，存放当前任务规划、子目标、工具返回，常驻 prompt 且随轮次刷新，可用 Redis 设 TTL 做跨轮暂存
- 长期记忆分 recall storage（对话流水）和 archival storage（抽取知识），MemGPT 用函数调用读写外部存储，以此突破上下文上限
- 分层的核心权衡在成本与一致性，长期记忆要异步写入并做近邻去重，避免每次推理都全量加载
- 工程上用分层缓存：热数据留窗口，温数据留 Redis，冷数据落向量库加 SQL 双写

面试官追问：

- 追问：上下文窗口变大后还需要工作记忆吗？怎么接：仍然需要，因为窗口再大也有成本与注意力稀释，工作记忆负责把当前子目标显式放在 prompt 前部，避免被长历史淹没，Redis 暂存跨轮中间结果还能省掉重复推理。
- 追问：长期记忆写入延迟怎么不阻塞主链路？怎么接：走异步双写队列，主链路只更新工作记忆并打写缓冲标记，落库交给后台 worker 完成，失败可重试，不影响当轮回复。

避坑：

- 别把所有历史全塞窗口，128K 看着大，注意力稀释后远端信息基本失效，该落库就落库
- 别把工作记忆当持久存储，进程重启就丢，跨轮关键态必须外置

6.2 记忆的存储与检索

题 2：向量记忆 vs 结构化记忆

考察点：语义检索与精确查询的适用边界

开口怎么答：

我不会二选一，通常两者并存，向量负责模糊召回，结构化负责精确过滤，关键看查询是语义相似还是属性命中。

回答框架：

- 向量记忆把文本嵌成 dense vector，用 ANN 索引（HNSW、IVF）在 Milvus 或 Qdrant 里做相似度检索，适合我上次聊过之类语义查询
- 结构化记忆存 SQL 或图数据库，字段可建索引，适合按用户 ID、时间范围、标签精确取数，延迟低且可解释
- 二者可混合，先向量召回候选再用结构化字段过滤，比如先语义找相关记忆再按时间窗裁剪
- 向量有召回噪声，需要重排，常用 bge-reranker 或交叉编码器；结构化维护成本低但要预先定义 schema
- 选型上长期知识沉淀走向量加图，业务状态走 SQL，会话摘要走向量加 TTL

面试官追问：

- 追问：向量检索召回了无关记忆怎么办？怎么接：加 rerank 阶段用小交叉编码器重排，并设置相似度阈值过滤低分，必要时回退到最近 N 轮原文兜底。
- 追问：结构化 schema 变动频繁怎么处理？怎么接：核心字段保持稳定，易变维度放 JSON 扩展列或走图数据库加边，避免频繁 DDL 阻塞写入。

避坑：

- 别指望向量做精确过滤，相似度近邻不等于属性命中，金额、状态这类必须走结构化
- 别忽视重排，只靠 ANN top-k 注入会让噪声记忆挤占窗口

题 3：长期记忆怎么存、怎么召回

考察点：持久化方案与检索链路设计

开口怎么答：

我落地的做法是写时分层、读时多路召回，存储上向量库加 SQL 双写，召回时先意图分类再选检索器。

回答框架：

- 写路径：对话结束抽取事实与偏好，结构化字段写 Postgres，语义块写 Qdrant，用 pgvector 也可统一；写入带 user_id、时间戳、来源
- 召回路径：先小模型做 query 分类，事实类走 SQL 精确查，经验类走向量召回 top-k，再 rerank 取前 n 条注入 prompt
- 借 Generative Agents 的检索打分思路，结合 recency、importance、relevance 三因子加权，避免只按相似度排序
- 控制注入量，通常 5 到 10 条记忆加摘要，超量会撑爆窗口并放大噪声，需要做预算分配
- 一致性靠幂等写入加版本号，重复记忆用 embedding 近邻去重，定期合并同一事实的不同表述

面试官追问：

- 追问：多用户并发下召回怎么避免串档？怎么接：所有记忆带 user_id 与 tenant_id 隔离，向量检索加元数据过滤，SQL 走行级约束，检索前强制绑定身份。

- 追问：记忆量到千万级索引还能撑吗？怎么接：靠分片与标量量化减内存，热用户记忆做本地缓存，冷数据下沉对象存储加惰性加载。

避坑：

- 别只做向量召回，没有身份隔离会串档，这是多租户 Agent 最常见的事故
- 别无脑全量注入，记忆越多不代表越好，噪声会压垮关键事实

6.3 上下文压缩与保真

题 4：对话历史压缩不丢关键信息

考察点：上下文压缩策略与信息保真

开口怎么答：

我一般不让历史原样进窗口，会按任务阶段做有损压缩，但要保留实体、决策和未决项，靠结构化摘要加要点回写。

回答框架：

- 摘要法：每轮或每 N 轮用 LLM 生成滚动摘要，存为 working memory，原对话落长期；MemGPT 的 conversation summary 就是这个思路
- 压缩法：LLMLingua 用小模型做 token 级压缩，压缩率常到 10 到 20 倍且保住任务准确率，适合省成本
- 结构化抽取：把关键信息抽成槽位如 entity、decision、todo，注入时只带槽位，丢弃闲聊与重复确认
- 保真手段：压缩前先抽取关键三元组，压缩后做事实一致性校验，丢项就回查原始流水
- 工程上设压缩触发阈值，比如累计 token 超窗口 70% 触发，冷启动保留最近 K 轮原文

面试官追问：

- 追问：压缩后模型忘了关键约束怎么兜底？怎么接：先把硬约束抽成不可压缩的规则槽位单独保留，压缩只针对闲聊与重复，再对压缩摘要做事实回查。
- 追问：LLMLingua 压缩会影响工具调用吗？怎么接：会在压缩前保留函数名与参数结构，只压自然语言部分，必要时对工具相关片段做白名单保护。

避坑：

- 别对所有内容一视同仁地压，硬约束和实体要白名单保护，压没了任务就跑偏
- 别只依赖摘要，摘要也会幻觉，关键决策要回写结构化槽位而非纯文本

6.4 记忆污染与遗忘治理

题 5：记忆污染与遗忘机制

考察点：错误记忆的防护与生命周期管理

开口怎么答：

记忆不是只写不删，我把它当带生命周期的存储，要有置信度、要有遗忘，否则错误和过期信息会反复污染决策。

回答框架：

- 污染来源：模型幻觉写入、用户口误、过时事实，需要在写入时打来源与置信度，召回时按置信度加权
- 遗忘机制参考 human-inspired forgetting，按访问频率与时间衰减降权，长期不用的记忆进入冷区或软删除
- 纠错靠冲突检测，新写入与旧记忆 embedding 相似但内容矛盾时触发人工确认或置信度投票
- 写时校验：关键决策类记忆要求工具返回或用户确认才落库，降低幻觉污染
- 定期 purge 脚本按 TTL 加访问计数清理，比如 90 天未访问且置信度低直接归档

面试官追问：

- 追问：用户明确说错了话但已写入怎么办？怎么接：保留写入溯源，新信息来时触发冲突检测，矛盾记忆标低置信并提示用户确认，确认后旧记忆降级为 archived。
- 追问：遗忘会不会误删重要记忆？怎么接：只对低置信加长期未访问的做软删除，重要记忆靠访问与人工置顶保活，删除可回溯一段时间。

避坑：

- 别让记忆只增不减，过期和错误记忆不清理，越用越蠢
- 别把模型生成内容当事实直接落库，没有来源和置信度的记忆是污染源头

6.5 用户画像沉淀

题 6：用户画像 / 偏好怎么沉淀

考察点：画像构建的增量更新与一致性

开口怎么答：

我把画像当成一张可演化的结构化表，不是一次性写死，每轮对话增量更新，关键偏好还要回灌确认避免猜错。

回答框架：

- 存储：画像主表放 SQL，一行一个用户，字段如语言偏好、行业、禁忌；语义偏好额外写向量便于模糊匹配
- 更新：对话中用抽取器识别偏好信号，写时做 upsert，相同维度新值覆盖旧值并保留变更历史
- 冲突处理：用户改口时按时间戳取最新，并标记 uncertain 让后续对话二次确认
- 注入策略：画像只取高置信字段进 system prompt，避免把猜测当事实强加给用户
- 隐私：敏感偏好加密存储，提供遗忘接口，满足 GDPR 类删除权，用户可要求清画像

面试官追问：

- 追问：画像和当前对话冲突以谁为准？怎么接：当前对话优先级最高，画像作为默认背景，冲突时当场偏好覆盖历史，并增量更新画像时间戳。

- 追问：画像泄露风险怎么控？怎么接：敏感字段加密且默认不进 prompt，只有显式需要才解密注入，提供一键清除接口满足删除权。

避坑：

- 别把低置信猜测当事实写进画像，一次误判会被反复强化
- 别忽视删除权，画像无清理接口迟早踩合规红线

6.6 长任务状态管理

题 7：State 在长任务里的管理方式

考察点：长程任务的状态机与恢复

开口怎么答：

长任务我最怕的是中途崩溃丢进度，所以状态必须外置成可持久化的状态机，每步落库，重启能续上。

回答框架：

- 状态外置：把 plan、subtask 状态、工具中间结果写 Redis 或 SQL，key 带 run_id，不依赖进程内存
- 用状态机建模，定义 pending、running、done、failed 等态，迁移靠显式事件，避免隐式推进
- 检查点：每完成一个子目标写 checkpoint，崩溃后从最近 checkpoint 重放，配合幂等工具调用防重复副作用
- 例子：LangGraph 的 checkpoint 把 graph state 持久化到 Postgres，支持 thread 级恢复与回退
- 超时与补偿：长任务加整体 TTL，failed 态触发补偿或人工介入，状态表保留 trace 便于复盘

面试官追问：

- 追问：子任务失败要不要整任务回滚？怎么接：按补偿而非回滚，failed 态触发该步补偿动作，已完成步骤保留，从 checkpoint 续跑，避免重复副作用。
- 追问：状态机和纯 ReAct 比优势在哪？怎么接：ReAct 全靠上下文隐式推进，长任务易丢步，状态机把进度外置且可观测，崩溃可恢复，也方便人工介入与审计。

避坑：

- 别把长任务进度只放内存，进程一挂全丢，必须每步落库
- 别用隐式推进代替显式状态迁移，出了事你根本不知道卡在哪一步

第七章 工程落地与评测（11 题）

7.1 成本与延迟优化

题 1：Agent 的成本结构（token / 调用 / 工具）

考察点：能拆解 Agent 线上推理成本并明确优化优先级。

开口怎么答：

我会把成本拆成 token、模型调用次数、工具调用三类来看，其中工具侧的检索和重试开销最容易被低估。我的经验是 token 往往占大头，但工具失败重试在复杂任务里能把账单直接翻倍。

回答框架：

- Token 成本分层：input token 通常比 output 便宜 3 到 4 倍，以 GPT-4o 计 input 约 2.5 美元每百万、output 约 10 美元每百万，长上下文 RAG 会把 input 成本推高，要做上下文裁剪和窗口压缩。
- 模型调用次数：多轮 ReAct 循环里一次任务可能产生 10 到 30 次 LLM 调用，用 7B 级小模型做路由和分类、只在规划类关键步骤上大模型，能砍掉近 40% 调用。
- 工具成本：向量检索按次计费、外部 API 如搜索有单价，工具失败重试会叠加成本，要给工具调用做失败熔断和预算上限。
- 隐性成本：长链路里的超时、重试、把大对象序列化后传参，这些在账单里看不到却吃掉算力，要做链路级归因。
- 优化手段：prompt caching 命中价可降到原价的十分之一（Anthropic 系）、结果缓存复用、用 tokenizer 预估做单次对话预算 guardrail。

面试官追问：

- 追问：你们怎么给单次对话设成本预算？怎么接：我会按任务类型挂 token 预算和调用次数上限，超了就降级到小模型或直接返回兜底答案，不让长任务失控。
- 追问：怎么判断该上大模型还是小模型？怎么接：用路由分类器按意图复杂度分流，简单问答走本地 7B 模型，规划类才上 70B 或 GPT 级别。

避坑：

- 只盯 token 单价而忽略工具调用的叠加成本。
- 没做调用预算上限，长任务失控烧钱。

题 2：延迟优化：流式 / 并行 / 缓存

考察点：能从链路角度降低 Agent 端到端延迟。

开口怎么答：

我把延迟拆成首字延迟和总耗时两块来优化，流式解决体验、并行和缓存解决总时长。我们线上靠这三招把 P95 从 8 秒压到了 3 秒以内。

回答框架：

- 流式输出：LLM 侧开 SSE 逐 token
  回传，工具结果也做成流式增量，前端边收边渲染，首字延迟压到 300 毫秒内。
- 并行工具调用：多个工具无依赖时并发执行，比如同时查订单和查物流，用 asyncio
  或线程池避免串行等待。
- 缓存分层：语义缓存对相似 query 命中历史答案、prompt caching
  让系统提示和长上下文命中缓存、工具结果缓存对相同入参直接返回。
- 预取与 speculative：模型生成中间步骤时预启下一个可能用到的工具，减少 idle 等待。
- 模型与链路精简：蒸馏小模型做第一步规划、减少 ReAct 步数，每少一轮调用省 500 毫秒到 1
  秒。

面试官追问：

- 追问：语义缓存命中率太低怎么办？怎么接：我会加 embedding
  相似度阈值加聚类去重，对高频意图单独做精确缓存，命中率低往往是 query 归一化没做好。
- 追问：流式下怎么保证最终答案完整可评测？怎么接：流式只负责展示，后端仍保留完整结构
  化输出和 trace，评测回放走完整链路而非截断流。

避坑：

- 为了快把有依赖的工具调用也强行并行，结果算错。
- 只做首字优化忽略总时长，用户等最终结果还是久。

7.2 并发控制与可观测性

题 3：并发与限流怎么设计

考察点：能设计保护下游和模型供应商的限流与并发控制。

开口怎么答：

我把它当成一个多级漏斗来做，从网关到模型 provider
到工具层各有各的限流。核心目标是别把上游 LLM 配额打爆，也别让单个用户拖垮全体。

回答框架：

- 网关层限流：按用户、按 API key 做令牌桶 token bucket，比如单用户 10
  QPS，超出直接返回 429。
- 模型 provider 侧：聚合全局并发上限，按各家 RPM 和 TPM 配额用信号量 Semaphore
  控制在飞请求数，超出排队或快速失败。
- 工具层隔离：不同工具独立限流和熔断，搜索工具挂了不影响本地计算，用 bulkhead
  隔离舱模式。
- 优先级队列：付费用户和高优任务插队，低优任务退避重试，配指数退避加抖动。
- 降级策略：限流触发时返回缓存答案或简化链路，配合 SLA 指标做自动降级开关。

面试官追问：

- 追问：限流和用户体验怎么平衡？怎么接：我用排队加进度提示代替直接拒绝，超阈值时短等待而非 429，并给高价值用户预留配额。
- 追问：多模型供应商怎么统一限流？怎么接：抽象一层 provider 适配，把各家的 RPM 和 TPM 换算成统一配额池，做跨供应商 failover。

避坑：

- 只在网关限流，模型侧并发没控，依然触发 provider 的 429 丢请求。
- 限流阈值写死，流量峰值时没留自动扩容和降级余量。

题 4：可观测性：trace / 日志 / 回放

考察点：能用标准方案让 Agent 行为可追踪、可复盘。

开口怎么答：

我觉得 Agent 不上可观测就等于裸奔，我们直接用 OpenTelemetry 把每一步 LLM 调用和工具调用打成一个 trace。出问题能回放整条链路，定位到底是模型抽风还是工具超时。

回答框架：

- Trace 标准：用 OpenTelemetry 的 span 模型，把一次对话拆成 plan、tool_call、llm_generate 等 span，parent-child 串联，导出到 Jaeger 或 LangSmith。
- 日志结构化：每一步记 input、output、token、耗时、模型版本，用 JSON 日志进 ELK，关键字段建索引便于检索 bad case。
- 回放机制：把线上 trace 落库，支持按 session_id 重跑，对比模型升级前后的输出差异。
- 指标体系：建 token 消耗、调用成功率、工具错误率、P95 延迟等 dashboard，LangSmith 和 Phoenix 都能直接看。
- 成本归因：trace 里带 user_id 和 task 标签，按维度分摊成本，定位烧钱的任务类型。

面试官追问：

- 追问：trace 数据量太大怎么存？怎么接：只采样保留全量错误和慢链路，正常链路按 1% 到 5% 采样，热数据进 OLAP、冷数据转对象存储。
- 追问：LLM 输出不确定，回放结果不一致怎么评？怎么接：回放时固定 temperature 和种子，并对比结构化字段而非纯文本，容忍表述差异。

避坑：

- 只记最终答案不记中间步骤，出问题全链路黑盒无法定位。
- trace 里塞了用户隐私明文，合规过不了。

7.3 评测框架与 LLM 评委

题 5：Agent 评测框架怎么搭（离线 + 线上）

考察点：能搭建覆盖研发到生产的完整评测闭环。

开口怎么答：

我会把评测分两半，离线跑回归保质量，线上跑监控防退化。离线用固定评测集加 LLM-as-judge，线上靠真实流量抽样加人工抽检。

回答框架：

- 离线评测集：构建覆盖核心意图、边界、对抗样本的标注集，每次模型或 prompt 改动都跑全量回归，门槛卡准确率。
- 自动评分：用 GPT-4 级模型做 LLM-as-judge（参考 JudgeBench），配 rubric 打分，复杂任务用 AgentEval 做多维度评估。
- 指标分层：任务成功率、工具调用正确率、答案忠诚度（RAGAS faithfulness）、安全性，分类别监控。
- 线上评测：影子流量 shadow 对新版本做无伤对比，再小流量灰度，真实指标达标才全量。
- 持续回归：把线上 bad case 自动回流成离线用例，防止同类问题复发，形成闭环。

面试官追问：

- 追问：离线高分线上翻车怎么解释？怎么接：通常是评测集分布和真实流量有偏，我会补线上采样进评测集，并加分布校验。
- 追问：judge 模型和被测模型同源会怎样？怎么接：容易互相包庇，我会换更强或不同家族的 judge，并保留人工校准。

避坑：

- 只用自动指标不看人工，分数虚高。
- 离线评测集长期不更新，和线上分布越漂越远。

题 6：用 LLM 评 LLM 的坑

考察点：清楚 LLM-as-judge 的偏差来源和缓解办法。

开口怎么答：

我踩过不少 judge 的坑，最典型的是位置偏好和长度偏好，长答案明明啰嗦却被打高分。我的做法是对比评测加 rubric，再拿人工抽检校准。

回答框架：

- 位置偏差：pairwise 评测里排前面的答案容易占优，要做正反双向各评一次取平均。
- 长度与 verbosity 偏差：越长的输出越容易被判质量高，要对答案做长度归一或要求 judge 只看事实。
- 自我偏好：judge 和被评同家族时打分宽松，换异源强模型或加 reference 答案约束。
- rubric 不一致：自由打分波动大，改成分维度 checklist 打分，提升可复现性。
- 校准与抽检：定期用人工标注集算 judge 的准确率和一致性（Cohen's kappa），偏差超阈就调 prompt 或换模型。

面试官追问：

- 追问：judge 成本太高怎么办？怎么接：用更小但够用的 judge 模型，只对边界样本上强模型，其余走规则或检索比对。
- 追问：怎么证明 judge 可信？怎么接：建一份带 gold label 的校准集，报告 judge 和人工的一致性指标，低于 0.8 不单独采用。

避坑：

- 直接把 judge 分数当绝对真理，没有人工校准。
- pairwise 不双向，顺序 bias 污染结论。

7.4 bad case 闭环与灰度发布

题 7：bad case 闭环怎么跑

考察点：能把线上问题转成可回归的改进资产。

开口怎么答：

bad case 不闭环就是白收集，我们做法是线上标记、自动聚类、回流评测集、改完跑回归。一个 case 从发现到进回归平均两三天。

回答框架：

- 收集入口：在对话末尾加反馈按钮，结合人工抽检和 judge 低分自动打标，统一进 bad case 库。
- 聚类归因：按错误类型（答非所问、工具调错、幻觉、越权）聚类，优先处理高频簇。
- 回流评测：每个 case 转成结构化测试用例，挂到对应任务评测集，成为回归红线。
- 修复验证：改 prompt 或工具后跑回归，确认该 case 通过且不拖累其他用例。
- 度量改进：跟踪 case 解决率、复发率，用趋势图看整体质量走向。

面试官追问：

- 追问：case 太多排不过来怎么取舍？怎么接：按影响面乘发生频率排序，高优簇先打，长尾簇攒一批批量处理。
- 追问：怎么防止修复一个坏另一个？怎么接：靠全量回归套件兜底，改动前后跑同一套用例，diff 异常立即告警。

避坑：

- 收集了 case 但没人归类，堆成死库。
- 只修表象不回流评测，同类问题反复出现。

题 8：生产环境灰度与回滚

考察点：能安全地把 Agent 改动推到生产并可控回退。

开口怎么答：

我倾向于小流量灰度加自动熔断，绝不上来全量。新 prompt 或新模型先 1% 流量，看核心指标平稳再逐级放大。

回答框架：

- 流量分层：按用户分桶或随机百分比灰度（1% 到 5% 到 20% 到 100%），配置中心动态调比例。
- 指标护栏：灰度期间盯成功率、延迟、成本、bad case 率，任一突破基线就自动缩量。
- 影子发布：新版本并行吃影子流量，只记结果不返回用户，验证无误再切真实流量。
- 一键回滚：保留上一版快照，配置开关秒级切回，回滚不依赖重新发版。
- 多版本共存：用 model registry 管理版本，A/B 对比用统计显著性检验确认收益。

面试官追问：

- 追问：回滚后用户上下文怎么处理？怎么接：会话状态按版本隔离，回滚时新会话走旧版，在飞会话允许自然结束，避免中途切换错乱。
- 追问：灰度指标波动怎么区分真问题？怎么接：设最小样本量和置信区间，低于阈值不决策，配合人工抽检排除噪声。

避坑：

- 灰度比例一上来就给太高，事故放大。
- 没留回滚开关，出问题只能重新发版救火。

7.5 安全治理

题 9：安全：提示注入 / 越权 / 数据泄露

考察点：能识别并防护 Agent 三类核心安全风险。

开口怎么答：

Agent 比普通 chatbot 危险在它能调工具，提示注入能直接指挥它干坏事。我的防护是输入隔离加最小权限加输出过滤三层。

回答框架：

- 提示注入防护：用 delimiting 标签隔离用户内容和系统指令，对不可信外部内容（网页、邮件）标记为 data 而非 instruction，参考 NIST 的指令层级做法。
- 最小权限：工具按角色授权，Agent 默认无高危操作权限，敏感工具需二次确认或人工审批。
- 越权访问控制：每次工具调用带用户身份校验，防止 A 用户借 Agent 读 B 用户数据，后端做资源归属校验。
- 数据泄露防护：输出侧加 PII 检测和脱敏，禁止把内部 system prompt 和工具 schema 回显给用户。
- 红队与监控：定期跑注入攻击样本（如 ignore previous instructions），对异常工具调用实时告警。

面试官追问：

- 追问：提示注入防不住怎么办？怎么接：把不可信内容严格当数据、关键动作加人工确认，并限制 Agent 能接触的系统边界。
- 追问：怎么检测隐蔽的数据泄露？怎么接：在输出网关做正则加模型扫描双保险，对含内部字段或长串 token 的返回直接拦截。

避坑：

- 把外部抓取内容当可信指令执行，被注入劫持。
- 工具权限一刀切放开，越权风险爆炸。

7.6 评测集构建与 SLA 守护

题 10：评测集怎么构建才不偏

考察点：能构建有代表性、低偏差的评测集。

开口怎么答：

评测集偏了，再高的分数都是假象。我建集的原则是覆盖真实分布、有人工 gold label、定期补线上长尾。

回答框架：

- 分布对齐：从真实流量采样而非手写，按意图占比分层抽样，避免只挑简单样例刷分。
- 多维度覆盖：包含正常、边界、对抗、多轮四类，对抗样本专门测注入和越权。
- 黄金标签：关键样本人工标注标准答案和评分 rubric，保证可复现，不靠主观临时判。
- 去污染：构建时和训练数据做重叠检测，防止测试题泄漏进训练导致的虚高。
- 动态更新：把线上 bad case 和新兴意图持续回流，季度级重审分布，防止概念漂移。

面试官追问：

- 追问：样本量不够怎么保证统计显著？怎么接：对核心指标做置信区间估计，样本少就聚焦高影响场景，并报告误差范围不夸精度。
- 追问：怎么发现评测集自身有偏？怎么接：看各子集通过率差异和线上表现 gap，gap 大说明集子没覆盖真实分布。

避坑：

- 评测集全靠工程师拍脑袋，分布和线上脱节。
- 多年不更新，模型换了新场景测不到。

题 11：SLA 怎么定、怎么守

考察点：能给 Agent 服务定合理 SLA 并建立守护机制。

开口怎么答：

SLA 不能拍脑袋，得看用户容忍度和链路上限。我们对外承诺 P95 延迟 5 秒、可用性 99.5%，守靠限流、降级和告警三件套。

回答框架：

- 指标定义：明确可用率、延迟分位（P50、P95、P99）、错误率、首次响应时间，写进服务等级目标 SLO。
- 链路预算：把总延迟预算拆给各阶段（LLM、工具、网络），任一段超预算就优化或降级。
- 限流保护：多级限流防过载，过载时保核心链路、牺牲非关键功能，避免雪崩。
- 降级与兜底：模型超时走缓存答案或简化链路，工具失败返回部分结果而非全失败。
- 告警与复盘：设 SLA 燃尽告警，超阈自动通知，月度复盘违约根因并改进。

面试官追问：

- 追问：SLA 和成本冲突怎么取舍？怎么接：用分层 SLA，付费用户高保障、免费用户宽松，把预算花在刀刃上。
- 追问：第三方模型抖动导致违约算谁的？怎么接：在 SLA 里区分依赖项，做多供应商 failover 和降级，把外部风险内化进预案。

避坑：

- SLA 定得比链路能力还高，永远违约。
- 只定不守，没有告警和降级，违约了才知道。

第八章 系统设计与案例（8 题）

8.1 垂直场景 Agent 设计

题 1：设计一个 Coding Agent（架构 + 工具 + 评测）

考察点：端到端代码生成 Agent 的分层架构、工具编排与闭环评测能力。

开口怎么答：

我一般会把它拆成三层来看，交互层负责把需求变成可执行的任务图，执行层在沙箱里真正跑代码，评测层做持续验证。我不建议一上来就堆大模型调用，得先把工具边界和反馈回路定清楚，否则 Agent 容易陷入自嗨式改代码。

回答框架：

- 架构上采用规划加执行的 ReAct 变体，规划器用 Claude 或 GPT-4o 级别的模型产出任务分解 DAG，执行器调用文件系统读写、shell 执行、单元测试运行三类工具，参考 SWE-agent 的 Agent-Computer Interface 设计动作空间，动作粒度控制在 diff 级而非整文件覆盖。
- 工具集必须跑在隔离沙箱里，我用 Docker 容器加资源限额，CPU 配 2 核、内存 4G、单次执行超时 60 秒，网络默认禁出网，只放行内网私有包源。代码执行产物落盘到临时卷，跑完即焚，避免污染宿主机。
- 检索能力要接代码库语义索引，用 RepoGraph 加 BM25 混合召回，先把相关函数签名和调用链喂给模型，再让它改。Devin 的核心思路就是长上下文加工具循环，而不是单次生成。
- 评测闭环分两层，静态层用 ruff 加 mypy 做语法和类型校验，动态层跑 pytest 用例，失败信息反向注入下一轮 prompt。我用 SWE-bench 的 resolved rate 当主指标，内部仓库目标定在 40% 到 55%，远超随机基线。
- 失败重试设上限为 5 轮，每轮带不同的检索上下文避免卡死，超过上限转人工 review 队列。成本上单次任务平均消耗约 12 万 token，配缓存后压到 7 万左右。

面试官追问：

- 追问：沙箱逃逸你怎么防？怎么接：我会叠三层，容器内核隔离加 seccomp 系统调用白名单，再加一个 eBPF 探针监听异常 fork 和文件写，命中规则直接 kill 容器并告警，网络层用 iptables 锁死出网。
- 追问：模型换更便宜的小模型行不行？怎么接：规划器可以下沉到 Claude Haiku 这类模型，但执行器改代码必须用强模型，我做过 A/B，弱模型在 SWE-bench 上 resolved rate 掉到 18%，不划算。

避坑：

- 别把所有逻辑塞进一个超长 prompt，任务图拆不开，模型一改就乱。
- 沙箱别给太松的权限，调试期图省事放开网络，上线就是安全事故。

题 2：设计一个客服 Agent（多轮 / 转人工 / 知识库）

考察点：对话状态管理、知识库检索增强与人工兜底策略的工程设计。

开口怎么答：

客服场景最怕两件事，一是答非所问让用户反复重复，二是该转人工时硬扛。我的做法是把对话当成一个有状态的状态机，知识库做第一道防线，置信度不够立刻交人，别让模型编。

回答框架：

- 多轮管理用对话状态跟踪 DST，把槽位槽值存进 Redis，TTL 设 30 分钟，用户意图用轻量意图分类模型（如 bge-m3 微调）识别，支持中途改槽和指代消解，靠 session_id 串联上下文。
- 知识库走 RAG 而非纯微调，文档切块用语义加滑动窗口混合，块大小 512 token 重叠 64，向量库用 Milvus 存 768 维句向量，召回 Top5 再让重排模型 Cross-Encoder 筛到 Top3，检索延迟压在 200 毫秒内。
- 转人工触发三条件，意图置信度低于 0.6、连续两轮用户表达不满、或命中敏感词清单，转之前把已收集槽位和对话摘要打包推给坐席系统，减少用户重复描述。
- 生成侧加护栏，答案必须带引用片段，禁止模型超出知识库范围臆测，用 self-check 提示词让模型标出不确定内容，不确定就走转人工。
- 指标上盯首解率、转人工率和 CSAT，目标首解率 70% 以上，转人工率控制在 15% 以内，超时 SLA 设为 3 秒响应。

面试官追问：

- 追问：知识库更新有延迟怎么办？怎么接：我接 CDC 增量同步，业务库变更通过 Kafka 推到索引管道，增量 embedding 平均 5 分钟内生效，热点文档走强刷接口秒级更新。
- 追问：用户骂人情绪激动怎么处理？怎么接：情绪识别单独挂一层，命中高负向先走安抚话术再转人工，同时给坐席打高优标签，避免机器人硬怼激化矛盾。

避坑：

- 别为了让首解率好看就压低转人工阈值，用户被机器人反复折腾更糟。
- 知识库脏数据直接进 RAG 会放大错误，上线前要做文档质量门禁。

题 3：设计一个长任务研究 Agent

考察点：超长任务的规划拆解、记忆管理与进度可观测设计。

开口怎么答：

长任务研究和写代码不一样，它周期长、中间状态多，我倾向于把它做成带检查点的工作流，而不是一个不停聊天的循环。关键要让它能被打断、能续跑，不然跑两个小时崩了全白干。

回答框架：

- 规划阶段用任务树，把研究目标拆成子问题节点，每个节点挂状态（待办、进行中、已完成、阻塞），用 LangGraph 的 state graph 持久化到 Postgres，支持 checkpoint 续跑，中断后从最近状态恢复。

- 检索层多源聚合，接 arXiv、网页搜索、内部文档三路，网页用 Jina Reader 转干净正文，去重靠 SimHash 相似度 0.85 阈值，避免同一观点反复进上下文。
- 记忆分三层，工作记忆放当前子问题上下文，情节记忆存已读文献摘要到向量库，语义记忆用一份 running notes 文档持续累积结论，每完成一个子问题就回写，防止长程遗忘。
- 进度可观测上，每个节点产出结构化 intermediate report，对外暴露进度条和日志，用户能随时插话改方向，Agent 收消息后重规划而非从头跑。
- 终止条件用目标达成校验加预算上限双保险，预算按子问题数量和 token 上限算，比如总预算 50 万 token，超了先出当前版本报告再问用户是否加预算。

面试官追问：

- 追问：研究结论互相矛盾怎么处理？怎么接：我让 Agent 在 running notes 里维护证据权衡表，标注支持和反对的来源，最终报告给结论置信度分级，不敢拍板的明确写存疑。
- 追问：中途用户改需求怎么办？怎么接：state graph 支持动态插入新节点并重排依赖，已完成的无关子问题保留，只重跑受影响分支，避免全量重做。

避坑：

- 别把所有文献一股脑塞进上下文，长程任务会爆窗口还拖慢速度。
- 进度不可见是最致命的，用户以为它卡死直接关掉，等于白跑。

8.2 Agent 与外部系统的集成边界

题 4：Agent 与外部系统的集成边界

考察点：工具接口抽象、权限与幂等设计、系统集成的安全边界把握。

开口怎么答：

集成边界我画一条很清楚的分界线，Agent 只负责决策和编排，真正落库、发消息、改配置这些写操作必须走受控的 API 网关。我不让模型直接拿数据库密码，那是灾难，工具是它伸出去的手，但手得戴手套。

回答框架：

- 工具抽象成统一 schema，每个工具定义名称、入参 JSON Schema、出参结构、副作用等级（只读、写、不可逆），模型只能看到 schema 和描述，拿不到底层凭证，凭证存在网关侧。
- 写操作强制幂等，每个调用带 idempotency_key，网关侧做去重，避免 Agent 重试造成重复下单或重复发短信，这是生产环境最容易翻车的地方。
- 权限走最小授权，工具调用带 scope 标签，发邮件和删数据的 scope 必须人工审批才能开，模型默认只有读和部分写权限，敏感动作触发二次确认。
- 边界隔离用 API 网关加限流，单 Agent 实例 QPS 限 20，突发熔断阈值 50，下游服务挂了返回结构化错误让 Agent 走降级而非疯狂重试。

- 审计上每次工具调用留痕，记录模型决策理由、入参、返回摘要，存到不可变日志，出了事能复盘到底是模型瞎调还是接口变了。

面试官追问：

- 追问：模型非要调用一个不存在的工具怎么办？怎么接：我会做 schema 校验拦截，未注册工具直接返回 unknown tool 错误并提示可用列表，同时记一次异常，连续多次就降级到受限模式。
- 追问：第三方接口字段变了怎么防？怎么接：网关层做响应结构断言，字段缺失或类型不符立刻告警并熔断该工具，不让脏数据灌进模型上下文误导后续决策。

避坑：

- 把数据库直连凭证塞进 prompt 是红线，一旦泄露全库裸奔。
- 忽略幂等设计，Agent 重试逻辑会在下游制造一堆重复副作用。

题 5：人机协作流程怎么设计

考察点：人在回路的介入时机、权限分级与协作效率权衡。

开口怎么答：

人机协作不是把人当备胎，而是把人放在决策的关键节点上。我的原则是高频低风险让 Agent 自动跑，低频高风险必须等人点头，这样既保效率又不失控。

回答框架：

- 介入分三档，全自动档用于只读和格式化产出，建议档用于写操作先给预览等人确认，审批档用于不可逆动作如退款和发公告，必须显式授权才执行。
- 协作界面要能解释，Agent 每次请求确认时附带决策依据和受影响对象清单，人能在界面上改参数再放行，而不是盲点同意，降低认知负担。
- 异步协作用任务看板，Agent 把待确认项推到队列，人批量处理，支持驳回并附理由，驳回触发 Agent 重规划，形成闭环而非单向指令。
- 信任校准靠数据，统计人工干预率和误干预率，如果人总在改 Agent 的结果，说明工具或 prompt 该迭代，别让人变成人工标注机器。
- 兜底责任链清晰，Agent 出错能追溯到哪一步、谁确认过，避免出事互相甩锅，这在金融和医疗场景尤其重要。

面试官追问：

- 追问：人一直不确认阻塞了怎么办？怎么接：我设 SLA 超时自动升级，普通任务等 30 分钟无人处理转值班，紧急任务 5 分钟未确认走电话告警，同时 Agent 给出默认安全动作如暂停而非继续。
- 追问：人确认了但还是错了算谁的？怎么接：责任按权限分级，全自动档平台担责，审批档由确认人担责，系统只保证把信息给全，决策权在谁手里锅就在谁身上。

避坑：

- 把人完全踢出循环省成本，高风险动作无人把关迟早出大事故。
- 确认界面信息太少，人只能瞎点，协作反而比纯人工还慢。

8.3 工程化与高可用架构

题 6：高并发 Agent 服务的架构

考察点：Agent 服务的水平扩展、异步调度与成本控制设计。

开口怎么答：

Agent 服务天然慢且贵，高并发不能靠堆机器硬扛，我一般把它拆成接入层、调度层和推理层三层，核心思路是把长耗时任务异步化，别让用户同步等。

回答框架：

- 接入层用网关做鉴权和限流，请求进来先落任务表返回 task_id，走轮询或 WebSocket 推进度，避免长连接占用，单网关扛 5000 QPS 没问题。
- 调度层用消息队列（Kafka 或 RabbitMQ）做任务缓冲，Worker 池消费，按模型类型分池，强模型池配少而精的实例，弱模型池配多的实例，削峰填谷。
- 推理层接多供应商 fallback，主用云端大模型，备用自建 vLLM 推理集群，配额满或超时自动切，成本敏感任务路由到本地 7B 量化模型，单卡能跑几十并发。
- 上下文缓存是省钱大头，用语义缓存，相似问题直接命中历史结果，命中率做到 30% 以上能把 token 成本砍掉近三成。
- 隔离上按租户做资源配额，单个租户异常不拖垮全局，配合 HPA 按队列长度弹性扩缩容，闲时缩到最小实例省钱。

面试官追问：

- 追问：某个大模型供应商全挂了怎么撑？怎么接：我备至少两家供应商加一套本地兜底，健康检查秒级探测，失败率超 10% 自动切流，本地模型顶上至少保证基础应答不中断。
- 追问：长任务占着 Worker 怎么办？怎么接：Worker 设最大执行时长，超时任务挂起到对象存储，释放 Worker 接新任务，续跑时从 checkpoint 恢复，资源利用率能拉到 80%。

避坑：

- 同步阻塞式调用在并发一上来就雪崩，连接池瞬间打满。
- 只接一家模型供应商，它抖一下你的服务就跟着抖。

题 7：Agent 的降级与兜底方案

考察点：故障场景下的优雅降级、安全护栏与可用性保障。

开口怎么答：

线上任何组件都会挂，我默认 Agent 永远处在会出问题的状态，所以降级不是可选项而是必答题。我习惯按故障层级准备三套兜底，从模型到工具到整体服务各管一段。

回答框架：

- 模型层降级，主模型超时 8 秒切备用模型，备用也挂就切本地小模型做基础应答，最差退回检索直出摘要，保证有东西回而不是报错。
- 工具层降级，关键工具挂了先走缓存结果，缓存也没有就标记该能力暂不可用并改写回答避开该动作，绝不假装执行成功骗用户。
- 护栏层常驻，输出过内容安全过滤器（如阿里云或自建分类器），命中违规直接拦截并返回安全话术，幻觉高风险场景加 fact-check 节点抽样校验。
- 服务层兜底，整体不可用直接返回友好降级页加重试建议，配合熔断组件如 Sentinel，错误率超阈值自动熔断保护下游。
- 可观测上把降级事件全量打点，监控降级触发频率，频率异常升高说明上游真有问题，该修上游而不是一直靠兜底撑着。

面试官追问：

- 追问：降级后回答质量下降用户投诉怎么办？怎么接：我在降级响应里明确告知当前为降级模式并建议稍后重试，降低预期管理成本，同时把降级事件推给值班，目标是在用户量低谷前恢复主链路。
- 追问：怎么防止降级被当成正常路径赖着不走？怎么接：降级是有时限的，监控持续探活主链路，恢复后自动切回并打恢复日志，人工也能在控制台强制复位，不让系统赖在兜底状态。

避坑：

- 降级时偷偷假装成功执行，用户以为办成了其实没办，比直接报错更伤人。
- 只做模型降级忘了工具降级，工具挂了模型再强也白搭。

8.4 项目落地与排期

题 8：从 0 到 1 落一个 Agent 项目怎么排期

考察点：Agent 项目的里程碑拆解、MVP 策略与风险管控能力。

开口怎么答：

我排期从来不是先把模型调到最优再上线，而是先跑通最矮的闭环验证价值，再逐步加能力。Agent 项目最大的坑是闷头做半年出一个华而不实的东西，市场早变了。

回答框架：

- 第一阶段两周做技术探针，选一个窄场景验证模型加工具能不能跑通闭环，产出一份可行性结论和成本测算，这一步砍掉不靠谱的方向最值钱。
- 第二阶段四周出 MVP，只做核心链路加最小工具集，人工在回路兜底，内部灰度小流量验证真实效果，指标先盯任务完成率和人工干预率。
- 第三阶段六周做工程化，补齐评测体系、监控告警、权限网关和高并发改造，把人工兜底逐步改成自动加审批，这时候才谈规模化。
- 排期里强制留 30% 缓冲应对模型抽风和接口变更，Agent 项目不确定性远高于普通后端，不留缓冲必延期。

- 风险清单前置，把模型幻觉、工具不稳定、成本超支列为头号风险，每个配应对预案和负责人，周会盯闭环而非只看进度条。

面试官追问：

- 追问：老板要一个月就要全量上线怎么顶？怎么接：我会把范围砍到单场景单链路，用人工兜底补能力缺口先上内部版，明确告诉老板全量是减配版，要全功能得加时间或加人，用数据说话而不是硬接。
- 追问：怎么判断 MVP
  该不该继续投入？怎么接：我设退出线，比如内部灰度两周任务完成率低于 50%
  且人工干预率高于 60%，说明方向或技术卡点没过，及时止损比硬撑强。

避坑：

- 排期零缓冲，模型一个抽风就把整个里程碑拖爆。
- 一上来追求全自动高并发，核心闭环都没验证就堆工程，本末倒置。

第九章 大厂面经真题复盘（30 题 · 网传整理）

> 以下题目来自网上公开面经（牛客、脉脉、腾讯云开发者、CSDN、字节剪映等）去重整理，非官方题库。每题配考察点，按厂按模块刷。刷的时候别只记答案，想清楚这道题在考你哪根能力轴。

9.1 字节（8 题）

题 1：Agent 怎么结合工具、知识和规划自主运行?

考察点：Agent 核心架构、闭环理解

开口怎么答：

我会把它拆成三层：规划层决定先干啥后干啥，工具层把规划落成对外的动作，知识层（RAG 或参数记忆）给每一步提供事实依据，三层靠一个循环串起来。

回答框架：

- 规划：用 ReAct 或 Plan-and-Execute
  让模型产出下一步目标和动作，复杂任务先出计划再逐步执行。
- 工具：把搜索、代码执行、数据库查询等封装成带 Schema 的工具，模型按语义匹配调用。
- 知识：需要外部事实时走 RAG
  检索注入上下文，需要长期偏好时走记忆检索，不靠模型凭空编。
- 闭环：每一步的观察结果回灌上下文，驱动下一轮决策，直到目标达成或触发停止条件。

面试官追问：

- 追问：规划和工具调用谁先谁后？怎么接：规划产出意图，工具执行意图，观察结果再回规划，是环不是线。
- 追问：知识层用 RAG 还是微调？怎么接：高频稳定知识走 RAG
  更灵活可更新，特定风格或能力才考虑微调。

避坑：

- 别把三个层讲成孤立模块，面试官要看的是它们怎么在循环里联动。
- 别只喊 RAG 和工具，漏掉规划层，那只是个问答系统不是 Agent。

题 2：多 Agent 架构下，主 Agent 与子 Agent 的通信链路怎么设计，异常怎么处理?

考察点：多智能体通信、容错设计

开口怎么答：

我的做法是主 Agent 当 Orchestrator，只做任务拆解和结果汇总，子 Agent 通过消息传递拿任务、回结果，通信链路要带超时和失败回退，不能让一个子 Agent 挂了拖垮整条链。

回答框架：

- 链路：主 Agent 用结构化消息（含 task_id、上下文、约束）派发，子 Agent 异步返回结果，主 Agent 按 task_id 聚合。
- 异常：每个子任务设独立超时，超时触发重试或降级；子 Agent 报错时主 Agent 决定换路径还是上报人工。
- 隔离：子 Agent 之间不直接耦合，都通过主 Agent 或共享状态总线通信，避免网状依赖难调试。
- 可观测：每条消息带 trace_id，出问题能完整回放派发到返回的整条链路。

面试官追问：

- 追问：子 Agent 返回格式乱了怎么办？怎么接：主 Agent 做结构化校验，失败走重试或规整，不把脏数据传给下一步。
- 追问：并发子任务怎么控？怎么接：用信号量或队列限制并发度，防止打爆下游工具或模型配额。

避坑：

- 别让子 Agent 互相直接调用，网状通信一出事定位能让你怀疑人生。
- 别省 trace_id，多 Agent 的 bug 全靠回放查，没链路日志就是瞎猜。

题 3：ReAct 的 Think / Action / Observation 在系统里怎么实现，tool_response 用哪个角色传回?

考察点：ReAct 工程实现、API 细节

开口怎么答：

Think 是模型一轮输出里的推理文本，Action 是模型吐出的工具调用请求，Observation 是工具执行结果。tool_response 在 OpenAI 体系里用 role 为 tool 的消息回传，带 tool_call_id 跟那次调用对上。

回答框架：

- Think：模型输出一段 reasoning（可隐藏也可显式），用于拆解当前局面。
- Action：模型返回 function_call 或 tool_use 结构，含函数名和参数。
- Observation：宿主执行工具，把结果包装成对应角色的消息塞回对话历史。
- 角色差异：OpenAI 用 role=tool 且带 tool_call_id；Anthropic 用 role=user 里的 tool_result block 关联 tool_use_id。
- 循环：把 Observation 追加进上下文，再让模型产出下一轮 Think/Action，直到不再请求工具。

面试官追问：

- 追问：Think 要不要回传给模型？怎么接：要，Think 是上下文的一部分，下一轮决策依赖它，只是前端可以不展示。
- 追问：一次多个 Action 怎么回传？怎么接：每个 tool 结果各带自己的 tool_call_id，分别回传，模型并行拿到所有 Observation。

避坑：

- 别说反了 tool 和 assistant 的角色归属，这是 API 调用的基本功。
- 别漏 tool_call_id 关联，缺了模型不知道这次 Observation 对应哪次调用。

题 4：长任务上下文爆炸怎么解，知不知道 ReSum 或常量工作空间?

考察点：长上下文管理、压缩策略

开口怎么答：

我会用分层加压缩，不让全部历史都堆在窗口里。听说过 ReSum
这类递归摘要思路，也实践过把早期内容压成常量大小的工作空间，只保留当前决策需要的。

回答框架：

- 分层：近期几步留原始上下文，更早的进长期记忆（向量库或数据库），用时检索。
- 递归摘要（ReSum 思路）：把历史分段摘要，再对摘要摘要，递归压成固定量信息。
- 常量工作空间：维护一份固定长度的 state 对象，每步读改写，把无限历史收敛成有限状态。
- 检索回填：需要早先细节时按语义从压缩记忆里捞回来，而不是靠窗口硬装。

面试官追问：

- 追问：压缩会丢关键信息吗？怎么接：会，所以关键事实进结构化 state
  保底，摘要只存脉络和决策点。
- 追问：窗口大到 200K
  还需要这套吗？怎么接：大窗口缓解成本问题但注意力仍退化，长任务还是得分层。

避坑：

- 别以为窗口大就万事大吉，成本和无意义冗余照样炸。
- 别只提一种方案，面试官想听你有分层、压缩、检索的组合拳。

题 5：幻觉怎么产生、怎么减轻，Structured Output 怎么控制输出?

考察点：幻觉治理、输出约束

开口怎么答：

幻觉主要来自模型对未知事实的脑补和长链路里错误 Observation
的累积。减轻靠检索约束加事实校验，控制输出靠 Structured Output 把格式和取值锁死。

回答框架：

- 成因：知识 cutoff、工具返回错误或缺失、多步推理累积偏差，都会让模型编。
- 减轻：RAG 给事实锚点、要求回答引用工具结果、关键结论加校验步骤。
- Structured Output：用 JSON Schema 或厂商原生 structured
  output，把字段和类型钉死，避免自由文本跑偏。
- 兜底：解析失败时重试或降级，格式错不进主流程。

面试官追问：

- 追问：Structured Output
  能保证内容不幻觉吗？怎么接：不能，它只保证格式，内容真实性还得靠检索和校验。
- 追问：怎么发现幻觉？怎么接：用事实校验步骤比对工具返回，或用评测集定期扫典型编造模式。

避坑：

- 别把 Structured Output 当防幻觉银弹，它管格式不管真假。
- 别只说「加检索」，要讲清检索失败时的校验和兜底。

题 6：Function Calling 和 RAG 的区别与联系，什么时候用哪个？

考察点：能力模块边界、场景判断

开口怎么答：

RAG 解决「模型不知道的事实」，Function Calling
解决「模型干不了的动作」，一个是补知识一个是补能力，实际系统里经常一起用。

回答框架：

- 区别：RAG 是检索增强，把外部知识塞进上下文让模型生成更准；Function Calling
  是让模型调外部工具，执行搜索、计算、写库等动作。
- 联系：RAG 的检索本身常通过 Function Calling
  触发，拿到结果再注入上下文，两者是上下游。
- 用 RAG：当问题依赖私有或时效知识、且答案在文档里能找到时。
- 用 Function Calling：当任务需要实时动作（查订单、发消息、跑代码）而非单纯生成时。

面试官追问：

- 追问：能不能只用 Function Calling 不用
  RAG？怎么接：能，让模型调搜索工具拿回原文再答，但 RAG 的检索质量更可控、可评估。
- 追问：两者会冲突吗？怎么接：不会，RAG 负责喂事实，Function Calling
  负责做动作，配合更完整。

避坑：

- 别把两者对立，面试官想听的是你理解它们互补。
- 别在只需实时动作的场景硬上 RAG，白白增加检索成本。

题 7：大规模 Agent 系统在多线程 / 多进程下的资源调度怎么设计？

考察点：并发架构、资源管理

开口怎么答：

核心是把有状态的会话和无状态的模型调用分开。会话状态放外部存储，模型调用走连接池和队列，用信号量控并发，别让每个请求都裸开一个进程。

回答框架：

- 状态外置：每个 Agent 会话的 state 存 Redis
  或数据库，进程重启不丢，进程本身无状态可水平扩展。
- 调用池：LLM 和工具调用走连接池，复用连接，限流保护下游。
- 队列削峰：高并发请求进消息队列，worker 按配额消费，避免瞬间打爆模型配额。
- 隔离：不同租户或优先级走不同队列，关键任务不被低优挤掉。

面试官追问：

- 追问：多进程怎么共享内存？怎么接：不共享，靠外部存储和消息总线，进程间只传 id
  和事件。
- 追问：模型配额打满怎么办？怎么接：队列背压加降级，超时任务走异步重试而非同步阻塞。

避坑：

- 别在进程内用全局可变状态存会话，扩不了还容易串号。
- 别忽视背压，模型配额比你想的更容易满。

题 8：Agent 记忆系统怎么设计，响应速度与推理精度怎么权衡？

考察点：记忆架构、性能权衡

开口怎么答：

我会分冷热：热记忆放上下文或 Redis
保速度，冷记忆放向量库保容量，检索时先查热再查冷，精度和速度靠分层检索平衡。

回答框架：

- 分层：当前任务上下文是热记忆，会话级状态进 Redis，长期偏好和知识进向量库。
- 检索顺序：先命中快的热层，未命中再走向量检索，避免每步都查库拖慢。
- 精度：冷层用向量召回加重排保证相关，热层用精确 key 读取保证即时。
- 权衡：检索太频拖累速度，太少又丢精度，按任务节奏调检索触发频率。

面试官追问：

- 追问：记忆太多会让模型变慢吗？怎么接：会，所以热层只留当前决策相关，其余下沉冷层按
  需取。
- 追问：怎么防止记忆过期？怎么接：加时间戳和衰减，旧记忆降权，关键事实显式更新覆盖。

避坑：

- 别所有记忆都查向量库，速度会崩。
- 别只讲容量不讲速度，面试官在考你工程权衡。

9.2 腾讯（8 题）

题 9：怎么设计一个多 Agent 系统，具体流程是什么？

考察点：多智能体架构设计

开口怎么答：

我会先定要不要多 Agent，要的话用
Orchestrator-Worker：一个总控负责拆解和汇总，多个专职 Worker
各管一摊，通信走消息和共享状态，配好超时和回退。

回答框架：

- 角色划分：按任务异质性拆
  Worker，比如一个做检索、一个做推理、一个做校验，避免职责重叠。
- 流程：Orchestrator 收任务、拆解子任务、派发给 Worker、收集结果、综合成最终输出。
- 通信：子任务用结构化消息传递，共享上下文走状态总线，不互相直接调。
- 保障：每步超时、失败重试、关键节点人工确认，整体可观测带 trace_id。

面试官追问：

- 追问：Worker 之间要互相对齐吗？怎么接：尽量不，靠 Orchestrator
  做汇总对齐，减少网状耦合。
- 追问：任务怎么拆？怎么接：按可独立验证的子目标拆，每个 Worker 的输出能单独测。

避坑：

- 别一上来就多 Agent，先论证单 Agent 为什么不够。
- 别让 Worker 自由互联，调试会爆炸。

题 10：跨境汇款场景下，Agent 超时 / 失败怎么应对，怎么保证资金安全？

考察点：高可靠场景、资金安全

开口怎么答：

资金场景我一句话原则：宁可卡住不可乱动。所有写操作走幂等加人工确认，超时按事务回滚或挂起待审，绝不自动重试可能重复扣款的动作。

回答框架：

- 幂等：每个汇款动作带唯一流水号，重复提交服务端去重，避免双扣。
- 超时处理：调用下游超时先挂起标记待确认，不盲目重试，由人工或对账补完。
- 人工确认：涉资金动作前 HITL 二次确认，模型只建议不执行。
- 对账兜底：异步对账任务定期扫异常流水，发现不一致告警。

面试官追问：

- 追问：模型误判要不要汇款怎么办？怎么接：模型只有建议权，执行权在带幂等校验的服务端，模型越权调不动。
- 追问：重试怎么防双扣？怎么接：重试带同一流水号，服务端幂等拦截。

避坑：

- 别在资金场景用自动重试，重复执行是事故。
- 别把模型当执行者，资金动作必须系统层硬卡。

题 11：RAG 知识库更新怎么做到不停服？

考察点：知识库运维、可用性

开口怎么答：

靠双索引热切换。新文档建好向量索引后，通过配置切换读取新索引，旧索引保留一段时间再下线，全程服务不中断。

回答框架：

- 双写/双索引：更新时先建新索引，验证无误再切流量，旧索引不立即删。
- 增量 vs 全量：小更新走增量入同一索引，大版本才用双索引切换。
- 一致性：切换瞬间用版本号标记，保证单次请求命中的是同一份索引。
- 回滚：新索引出问题秒切回旧索引，不影响线上。

面试官追问：

- 追问：增量更新会锁表吗？怎么接：用异步建索引加读写分离，更新不阻塞查询。
- 追问：Embedding 模型换了怎么办？怎么接：必须全量重建并双索引切换，不能混用不同模型向量。

避坑：

- 别直接原地改线上索引，一旦建错全量检索崩。
- 别忽略 Embedding 版本一致性，混模型向量检索全乱。

题 12：千轮上下文关联怎么实现，对话摘要算法具体怎么做？

考察点：超长对话、历史压缩

开口怎么答：

千轮不可能全留窗口，我会做滚动摘要加关键事件抽取，把对话压成「摘要加近期原文加结构化事实」，需要早先细节时再检索原始记录。

回答框架：

- 滚动摘要：每 N 轮触发一次摘要，把旧轮次压成脉络，保留决策点和待办。
- 事件抽取：用模型抽关键事实（订单号、偏好、承诺）进结构化档案，精确可查。
- 分层留存：近期原文留窗口，更早的进摘要和档案，原始日志落库可回溯。
- 检索：用户问早先内容时，先查档案和摘要，缺失再翻原始日志。

面试官追问：

- 追问：摘要会越来越长怎么办？怎么接：对摘要再摘要，或只保留最近 K 轮摘要加全局事实表。
- 追问：摘要出错影响后续吗？怎么接：关键事实走结构化档案而非摘要文本，降低被摘要带偏的风险。

避坑：

- 别把所有轮次塞窗口，成本和效果都会崩。
- 别只靠摘要，关键事实要结构化保真。

题 13：为什么 Agent 要分层而不是单个 Agent 处理所有任务，分层收益是什么？

考察点：架构分层动机

开口怎么答：

单 Agent 处理所有任务，上下文会无限膨胀、职责糊成一团、没法独立评测。分层把异质子任务拆给专职模块，每个都更准、更可控、更好测。

回答框架：

- 上下文可控：每层只装自己需要的上下文，避免全局污染。
- 职责清晰：检索、推理、校验各管一摊，出问题定位快。
- 可评测：每个子模块能单独跑单测和评测，不用端到端黑盒。
- 可控性：分层后能在关键层加审批和回退，整体更稳。

面试官追问：

- 追问：分层会不会更慢更贵？怎么接：会多几跳，但准确率和可维护性换回来，简单任务不必分太多层。
- 追问：层之间怎么传上下文？怎么接：用结构化消息传递必要信息，不把整段历史层层搬运。

避坑：

- 别为分层而分层，简单任务一层就够。
- 别让层间传全量历史，那和单 Agent 没区别。

题 14：ReAct 模式具体怎么落地，和直接调用工具相比优势在哪？

考察点：ReAct 落地、对比

开口怎么答：

落地就是让模型交替产出推理和工具调用，宿主执行工具把结果回灌，循环到能答。比直接调工具强在模型能根据中间结果动态改路，不是一条道走到黑。

回答框架：

- 落地：系统提示给工具和示例，模型输出 Thought 加 Action，宿主执行后把 Observation 回灌，再下一轮。
- 优势（对比直接调）：直接调是固定流程，ReAct 能看中间结果纠偏，适合不确定路径的任务。
- 约束：必须加最大步数和停止条件，否则易循环。
- 变体：复杂任务可先 Plan 再 ReAct，兼顾规划和灵活。

面试官追问：

- 追问：ReAct 比直接调工具慢多少？怎么接：多几轮生成，但任务成功率提升通常值回票价，简单任务还是直接调。
- 追问：Thought 一定要输出吗？怎么接：可省，但保留能提升复杂任务准确率和可调试性。

避坑：

- 别神化 ReAct，简单固定流程直接调更省。
- 别漏停止条件，循环是生产事故源头。

题 15：RAG 全链路：chunk 设计、Embedding 选型、ReRank 微调怎么搞?

考察点：RAG 工程全栈

开口怎么答：

我会从数据特性倒推：chunk 按语义边界切、Embedding 选领域适配的、召回后用 ReRank 精排，ReRank 用业务标注数据微调才能贴合场景。

回答框架：

- chunk：按标题层级或句子边界切，重叠保留上下文，避免切断关键句；长度看模型窗口和查询粒度。
- Embedding：通用场景用成熟模型，垂直领域用领域语料微调或选领域向模型。
- ReRank：双塔召回后用 cross-encoder 重排，用点击或人工标注的正负对微调，显著提升 top 准确率。
- 评估：用召回率和终局答案准确率双指标，别只看向量召回。

面试官追问：

- 追问：chunk 太大太小各有什么问题？怎么接：太大引入噪声、太小丢上下文，按查询粒度调，常用 256 到 512 token 起步实验。
- 追问：ReRank 必须微调吗？怎么接：不一定，通用 ReRank 够用就别微调，有标注数据且效果卡脖子再微。

避坑：

- 别只调 Embedding 忽略 ReRank，精排往往更影响体验。
- 别用通用 chunk 一刀切，文档结构不同切法不同。

题 16：Agent 端到端成功率怎么量化，线上最难监控的指标是什么?

考察点：Agent 评测、线上监控

开口怎么答：

成功率得按任务定义「完成」来量化，比如是否产出可用结果、是否触发人工、用户是否满意。最难监控的是「静默失败」：答案看着通顺但其实错了，指标上看不出。

回答框架：

- 量化：用任务级标签（成功/失败/转人工/用户驳回）统计，而非只看接口 200。
- 分层指标：工具调用成功率、步骤完成率、最终采纳率，分层看瓶颈。
- 最难监控：幻觉和逻辑偏差，输出格式正常但内容错，需要采样评测或用户反馈兜底。
- 监控手段：trace 抽样加 LLM-as-judge 定期扫，结合用户负反馈闭环。

面试官追问：

- 追问：成功率定太高会不会逼模型保守？怎么接：分档看，区分「完全成功」和「可用需改」，别只盯一个数字。
- 追问：怎么抓静默失败？怎么接：关键路径加事实校验，定期抽样本让模型互评或人工抽检。

避坑：

- 别只看接口成功率，那掩盖了大部分真实失败。
- 别忽视静默失败，它最伤用户信任。

9.3 阿里（7 题）

题 17：大模型 Agent 各核心模块的功能、难点和联动逻辑是什么？

考察点：Agent 全局架构

开口怎么答：

核心模块就四块：规划、记忆、工具、执行。规划决定干什么，记忆提供上下文，工具落地动作，执行把结果回灌规划，难点全在模块间的状态和错误传递。

回答框架：

- 规划模块：拆目标、定步骤，难点在长任务里的目标漂移。
- 记忆模块：短期上下文加长期存储，难点在压缩不丢关键。
- 工具模块：注册、调用、容错，难点在参数校验和失败降级。
- 联动：执行结果进记忆、驱动下一轮规划，任何一环出错都会放大，所以每环都要可观测。

面试官追问：

- 追问：哪个模块最难？怎么接：多数项目卡在记忆和工具容错，规划反而是最成熟的。
- 追问：模块间用什么解耦？怎么接：用统一的上下文对象和事件总线，模块不直接依赖彼此实现。

避坑：

- 别只列模块名，面试官要看联动和难点。
- 别把 Agent 讲成黑盒，要能拆开讲每环责任。

题 18：短期 / 长期记忆的存储、更新、检索机制分别怎么设计？

考察点：记忆系统设计

开口怎么答：

短期记忆走上下文和 Redis，快但易失；长期记忆走向量库加结构化表，慢但容量大。更新靠事件触发，检索先热后冷分层走。

回答框架：

- 短期：当前会话上下文在内存或 Redis，按轮次追加，超长压缩。
- 长期：用户偏好和知识进向量库加关系表，带时间戳。

- 更新：关键事件（用户改需求、确认事实）触发写入或覆盖，不是全量刷新。
- 检索：先查 Redis 热层，未命中走向量召回加重排，保证速度和精度。

面试官追问：

- 追问：长期记忆怎么防脏数据？怎么接：写入带来源和置信度，冲突时按时间和权重裁决。
- 追问：检索慢怎么优化？怎么接：热层前置加缓存，向量库加 IVF 等索引提速。

避坑：

- 别把所有记忆都堆向量库，热数据该进快存。
- 别忽略写入来源，脏记忆比没记忆更糟。

题 19：推理中的「推理断层」「结果与目标偏离」怎么解决？

考察点：规划纠偏、目标对齐

开口怎么答：

断层多半是步骤间缺乏状态传递，偏离是没持续对目标。我的解法是显式 state
加周期性自检：每几步让模型判断是否还朝着目标，偏离就回滚检查点。

回答框架：

- 断层：用结构化 state 在步骤间传关键事实，不靠模型自己记上下文。
- 偏离：每 N 步插入一个评估步，模型对比当前进展和原始目标，偏差超阈值就纠偏。
- 回滚：保留检查点，纠偏时回到最近一致状态重走，不一路错下去。
- 目标锚定：把原始目标拆成可验证的子目标，每步对照子目标是否达成。

面试官追问：

- 追问：自检步会不会拖慢？怎么接：只在长任务或关键节点触发，短任务不插。
- 追问：怎么定义偏离？怎么接：用子目标完成度加模型自评，超阈值告警而非自动大改。

避坑：

- 别依赖模型自觉不跑偏，要系统层加锚点。
- 别等任务结束才发现偏了，中期纠偏成本低得多。

题 20：Function Calling 完整流程怎么跑通，返回 JSON 不规范怎么兜底？

考察点：Function Calling 全流程、容错

开口怎么答：

流程是：系统提示声明工具 Schema，模型输出调用请求，宿主校验参数、执行、把结果用 tool 角色回传。JSON 不规范就校验报错、要求模型重生成，或走规则兜底解析。

回答框架：

- 声明：用 JSON Schema 描述每个工具的参数，写清类型和约束。
- 调用：模型返回 function_call，宿主先做参数校验，缺字段或类型错就退回重生成。
- 执行：校验通过才执行，结果包装成 tool 消息回传，带 tool_call_id。

- 兜底：JSON 解析失败先尝试修复（补引号、截断），再不行退 LLM 重生成，仍失败走降级不卡死。

面试官追问：

- 追问：模型老填错参数怎么办？怎么接：在工具描述里加示例和约束说明，必要时加 few-shot。
- 追问：并行调用出错一个怎么处理？怎么接：单点失败不影响其他，汇总时标记失败项让模型决定重试或绕开。

避坑：

- 别信任模型输出的 JSON，必做参数校验。
- 别让解析失败卡死主流程，兜底降级是生产基本要求。

题 21：多 Agent 协作常见的组织模式有哪些？

考察点：多智能体模式

开口怎么答：

常见就三类：流水线（顺序接力）、Orchestrator-Worker（总控派发）、黑板（共享空间协作）。选哪个看任务耦合度和要不要并行。

回答框架：

- 流水线：A 出结果给 B 再给 C，适合步骤强依赖、可串行。
- Orchestrator-Worker：总控拆任务派给专职 Worker，适合异质子任务并行。
- 黑板模式：各 Agent 读写共享黑板，适合需要反复协商的开放任务。
- 选型：依赖强走流水线，异质并行走 Orchestrator，反复协商走黑板。

面试官追问：

- 追问：黑板模式怎么防冲突？怎么接：黑板加版本和锁，写前读最新状态，避免覆盖。
- 追问：流水线卡一环怎么办？怎么接：每环超时降级，卡住环节走兜底输出不阻塞整条链。

避坑：

- 别张口就多 Agent，先说清任务适合哪种模式。
- 别混用模式，耦合和调试成本会失控。

题 22：一万篇长文档构建 RAG 知识库怎么落地？

考察点：大规模 RAG 工程

开口怎么答：

核心是切分、索引、检索三层都要能扛量。我会先做清洗和结构化切分，建分层索引，召回后重排，再加热点缓存，不能一把梭全量向量。

回答框架：

- 预处理：去重、去噪、按结构切 chunk，保留标题层级做上下文。

- 索引：向量库分片加 IVF/HNSW 索引，元数据处理过滤（时间、来源）缩小召回面。
- 召回：向量加关键词混合检索，再 cross-encoder 重排提精度。
- 提效：高频查询缓存答案，新文档异步入索引不影响查询。

面试官追问：

- 追问：一万篇怎么保证召回不漏？怎么接：混合检索加元数据过滤，定期用评测集扫召回率。
- 追问：更新一篇要不要全量重建？怎么接：不用，单篇增量入索引，只动相关分片。

避坑：

- 别全量一把梭，更新和查询都会卡死。
- 别只看向量召回，混合检索在大库上更稳。

题 23：模型微调（LoRA / DeepSpeed ZeRO）显存怎么估算、做了哪些优化？

考察点：微调工程、显存优化

开口怎么答：

显存大致是模型参数、优化器状态、梯度、激活四块之和。LoRA
只训低秩矩阵，参数量砍到个位数百分比；DeepSpeed ZeRO
把优化器状态分片到多卡，显存随卡数线性降。

回答框架：

- 显存构成：权重（fp16 约 2 字节每参数）、优化器（Adam 约 12
  字节每参数）、梯度、激活。
- LoRA：冻结底座，只训 A/B 两个低秩矩阵，可训练参数常降到原 1% 以下，显存大降。
- ZeRO：ZeRO-2 分片优化器加梯度，ZeRO-3 再加权重分片，多卡显存近似除以卡数。
- 优化：混合精度、梯度检查点省激活、offload 把状态放 CPU 兜底。

面试官追问：

- 追问：单卡 24G 能微调多大模型？怎么接：全参基本只能小模型，上 LoRA 加 ZeRO-offload
  能碰 7B 到 13B 级别。
- 追问：LoRA 秩怎么定？怎么接：从小秩起实验，看下游任务收益，不是越大越好。

避坑：

- 别只背公式，要讲清每块显存对应什么。
- 别忽略激活显存，长序列下它能超过参数本身。

9.4 美团（7 题）

题 24：美团智能客服 Agent 怎么设计多轮对话流程？

考察点：对话系统、多轮设计

开口怎么答：

我会把流程拆成意图识别、状态跟踪、工具调用、回复生成四段，每轮更新会话状态，该查的查该转人工的转，不让模型自由发挥乱承诺。

回答框架：

- 意图识别：每轮判断用户要什么，定位到对应业务流程。
- 状态跟踪：用结构化 state 记录订单号、当前步骤、已收集信息，避免重复问。
- 工具调用：查订单、改地址等动作走工具，结果回灌再生成回复。
- 转人工：置信度低或遇复杂纠纷，主动转人工并带上上下文，不甩给用户重说。

面试官追问：

- 追问：怎么判断该转人工？怎么接：意图模糊超两轮、或涉资金纠纷、或用户明确要求，就转。
- 追问：重复提问怎么避免？怎么接：state 里记已收集字段，生成前查 state 不重复问。

避坑：

- 别让模型瞎承诺退款，涉钱动作必须工具加审批。
- 别丢上下文转人工，用户最烦重说一遍。

题 25：会话状态怎么跟踪，用户意图模糊时怎么处理？

考察点：状态管理、歧义处理

开口怎么答：

状态用结构化对象跟踪，每轮读写。意图模糊时我先澄清再行动，用多选或反问收敛，不拿低置信猜测去调工具。

回答框架：

- 跟踪：state 存用户 id、当前场景、已确认事实、待收集槽位。
- 模糊处理：意图置信度低就主动澄清，给选项而非开放反问。
- 槽位填充：差什么信息就问什么，问完再调工具，不在信息不全时盲动。
- 兜底：多次澄清仍模糊就转人工，带 state 过去。

面试官追问：

- 追问：澄清会不会惹烦用户？怎么接：用选项式澄清比开放提问轻，且只在必要槽位问。
- 追问：state 丢了怎么恢复？怎么接：关键 state 落库，会话恢复时先读回，不靠模型记忆。

避坑：

- 别在意图模糊时硬调工具，容易错动作。
- 别只靠模型记状态，落库才稳。

题 26：工具异常（超时 / 接口失败）怎么容错？

考察点：工具容错、稳定性

开口怎么答：

三层：调用前限流加超时，调用中重试加降级，调用后校验结果。一个工具挂了不能拖垮整个 Agent，要有替代路径或兜底回答。

回答框架：

- 超时：每个工具设独立超时，超了走降级而非阻塞。
- 重试：瞬时错误（5xx、网络抖）指数退避重试，幂等接口才重试写操作。
- 降级：工具不可用就走近似方案（缓存答案、简化流程）或坦诚告知能力边界。
- 隔离：工具异常被捕获，不抛到主流程外导致整个 Agent 崩。

面试官追问：

- 追问：重试会不会雪崩？怎么接：退避加限流，且只重试读操作，写操作靠幂等不盲目重。
- 追问：降级答案不准怎么办？怎么接：标注「部分能力暂不可用」，不编，用户能接受诚实。

避坑：

- 别让一个工具挂拖垮全链路，隔离是底线。
- 别对写操作盲目重试，幂等是前提。

题 27：多知识库协同冲突怎么解决？

考察点：多源知识、冲突仲裁

开口怎么答：

先给每个知识库打来源权重和时效标签，召回后按权重和置信度仲裁，冲突时优先高权重或最新源，并把冲突暴露给人而非默默选边。

回答框架：

- 标注：每个库带来源、时效、置信度元数据。
- 仲裁：冲突时按权重加时间裁决，高权重新鲜源胜出。
- 暴露：关键冲突（如政策矛盾）标红提示人工，不私自定论。
- 路由：查询时先用元数据过滤到最相关库，缩小冲突面。

面试官追问：

- 追问：两个库权重一样怎么办？怎么接：优先时效，仍平就提示人工，不让模型随机选。
- 追问：怎么防过期库干扰？怎么接：入索引带生效时间，过期库降权或下线。

避坑：

- 别让模型在冲突里随机选，那是最危险的。
- 别忽略来源元数据，没标签没法仲裁。

题 28：多用户并发打 API 卡死，架构层面怎么设计？

考察点：高并发、限流架构

开口怎么答：

核心是队列加池加限流。请求进消息队列削峰，模型调用走连接池和信号量控并发，按租户限流，后端满就背压不硬扛。

回答框架：

- 队列削峰：突发流量进 MQ，worker 按配额消费，平滑打到模型。
- 连接池：复用 LLM 和工具连接，避免每次建连开销。
- 限流：按租户或用户配额限流，超限返回排队或降级。
- 背压：下游满时上游停发，不堆积压垮内存。

面试官追问：

- 追问：队列积压怎么处理？怎么接：动态扩 worker，设任务超时，过期任务通知用户重试。
- 追问：怎么防单用户打爆？怎么接：单用户并发上限加令牌桶，超了直接拒。

避坑：

- 别让请求直冲模型，没队列一峰值就满。
- 别省限流，单用户就能拖垮整体。

题 29：为什么用 LangGraph、有什么缺点、是不是过度设计？

考察点：框架选型、批判性

开口怎么答：

我用 LangGraph 主要图状态好管、可回退、可观测，适合多步有环的 Agent。缺点是学习曲线陡、调试相对重，简单线性 Agent 用它确实过度设计。

回答框架：

- 优点：有向图建模状态流转，支持循环和分支，内置 checkpoint 可暂停恢复，trace 清晰。
- 缺点：概念多（node、edge、state），简单场景上手成本高，抽象层带来调试负担。
- 过度设计判断：单线固定流程用它就重了，裸写循环或轻框架更合适。
- 替代：简单任务用原生 API 或轻量封装，只有复杂有状态才上 LangGraph。

面试官追问：

- 追问：和 AutoGen 比呢？怎么接：AutoGen 偏多 Agent 对话，LangGraph 偏可控状态图，按要不要强状态选。
- 追问：生产敢用吗？怎么接：敢，它的 checkpoint 和可控性正是生产需要的，前提是团队啃得动抽象。

避坑：

- 别一味吹框架，要讲清缺点。
- 别在简单场景硬上，过度设计是真实坑。

题 30：Agent 的编排层 / 模型层 / 工具层 / 观测层怎么拆，失败怎么回滚？

考察点：系统分层、回滚

开口怎么答：

四层各管一摊：编排管流程、模型管推理、工具管动作、观测管可观测。回滚靠编排层的checkpoint，失败回到上一个一致状态重走，观测层负责定位。

回答框架：

- 编排层：用状态图（如 LangGraph）定义步骤和分支，管流转。
- 模型层：封装 LLM 调用，管模型选择、重试、降级。
- 工具层：注册和执行工具，管参数校验和容错。
- 观测层：trace、日志、指标，管出问题能回放定位。
- 回滚：编排层在关键节点存 checkpoint，失败按 trace 回到检查点重走，不从头来。

面试官追问：

- 追问：回滚会不会重复副作用？怎么接：写操作走幂等，回滚只重走安全步骤，副作用步骤靠幂等重放。
- 追问：四层怎么解耦？怎么接：层间用事件和结构化消息通信，不互相直接依赖实现。

避坑：

- 别把四层糊一起，出问题没法定位。
- 别回滚不带幂等，重复副作用更糟。

能力维度速查表

这份表按能力轴把 100 道题重新索引了一遍。刷题前先扫一遍，看哪根轴发虚，直接跳到对应章节集中补。每道题标了出处章节和题号，方便回正文查完整回答框架。

一、基础原理（认知 Agent 的底层逻辑）

- 认知篇·题1：面试官怎么判断你真做过 Agent
- 认知篇·题2：初级、中级、高级的能力分水岭
- 认知篇·题3：这 100 题怎么用效率最高
- 基础原理·题1：什么是 Agent，跟普通 LLM 应用差在哪
- 基础原理·题2：ReAct 范式：推理与行动的循环
- 基础原理·题3：规划与反思：怎么决定下一步、怎么纠错
- 基础原理·题4：工具调用 / Function Call 的本质
- 基础原理·题5：上下文与状态管理：长任务怎么不丢信息
- 基础原理·题6：确定性 vs 自主性：什么时候该收住
- 基础原理·题7：失败模式（循环 / 幻觉 / 卡死）怎么排查
- 基础原理·题8：单 Agent 能解决，为什么别上多智能体
- 基础原理·题9：可控性怎么保证
- 基础原理·题10：最小可用 Agent 怎么搭

二、框架与选型（知道该用什么、为什么）

- 框架选型·题1：LangChain vs LangGraph 适用边界
- 框架选型·题2：AutoGPT / MetaGPT / CrewAI 设计哲学对比
- 框架选型·题3：自研 vs 开源框架的权衡
- 框架选型·题4：Dify / Coze 类低代码平台能走到哪
- 框架选型·题5：多智能体编排框架怎么选
- 框架选型·题6：Agent 记忆模块的主流实现
- 框架选型·题7：开源模型跑 Agent 的可行性
- 框架选型·题8：怎么给技术主管讲清选型理由

三、RAG 与知识增强（Agent 的事实底座）

- RAG·题1：RAG 在 Agent 里的角色（不只是问答）
- RAG·题2：向量库选型：Milvus / Qdrant / pgvector
- RAG·题3：Chunking 策略对检索效果的影响
- RAG·题4：混合检索（向量加关键词）怎么落地
- RAG·题5：GraphRAG 解决了什么、代价是什么

- RAG·题6：检索结果怎么喂给 Agent 做决策
- RAG·题7：RAG 评估：召回率之外看什么
- RAG·题8：知识库实时性怎么保证
- RAG·题9：检索失败时的兜底策略
- RAG·题10：多源知识冲突怎么处理

四、工具调用与 MCP（让 Agent 长出手脚）

- 工具调用·题1：Function Call 的 JSON Schema 怎么设计才稳
- 工具调用·题2：并行工具调用的并发与依赖处理
- 工具调用·题3：工具调用失败的重试与降级
- 工具调用·题4：MCP 协议解决了什么、和 Function Call 区别
- 工具调用·题5：怎么给 Agent 设计一套工具集
- 工具调用·题6：工具权限与安全边界
- 工具调用·题7：长链路工具调用的可观测性
- 工具调用·题8：工具结果过大怎么截断 / 压缩

五、多智能体协作（当单 Agent 不够时）

- 多智能体·题1：什么时候该上多智能体，什么时候是过度设计
- 多智能体·题2：Orchestrator-Worker 模式怎么落地
- 多智能体·题3：多智能体通信：消息 / 共享状态 / 黑板
- 多智能体·题4：角色分工与职责边界怎么定
- 多智能体·题5：多智能体间的冲突与一致性
- 多智能体·题6：人类在回路（HITL）怎么嵌
- 多智能体·题7：多智能体的成本与延迟怎么控
- 多智能体·题8：多智能体调试为什么更难

六、记忆与状态（Agent 的脑子）

- 记忆·题1：短期 / 工作 / 长期记忆的区分
- 记忆·题2：向量记忆 vs 结构化记忆
- 记忆·题3：长期记忆怎么存、怎么召回
- 记忆·题4：对话历史压缩不丢关键信息
- 记忆·题5：记忆污染与遗忘机制
- 记忆·题6：用户画像 / 偏好怎么沉淀
- 记忆·题7：State 在长任务里的管理方式

七、工程落地与评测（从 demo 到生产）

- 工程评测·题1：Agent 的成本结构（token / 调用 / 工具）
- 工程评测·题2：延迟优化：流式 / 并行 / 缓存
- 工程评测·题3：并发与限流怎么设计
- 工程评测·题4：可观测性：trace / 日志 / 回放
- 工程评测·题5：Agent 评测框架怎么搭（离线加线上）
- 工程评测·题6：用 LLM 评 LLM 的坑
- 工程评测·题7：bad case 闭环怎么跑
- 工程评测·题8：生产环境灰度与回滚
- 工程评测·题9：安全：提示注入 / 越权 / 数据泄露
- 工程评测·题10：评测集怎么构建才不偏
- 工程评测·题11：SLA 怎么定、怎么守

八、系统设计与案例（综合实战）

- 系统设计·题1：设计一个 Coding Agent（架构加工具加评测）
- 系统设计·题2：设计一个客服 Agent（多轮 / 转人工 / 知识库）
- 系统设计·题3：设计一个长任务研究 Agent
- 系统设计·题4：Agent 与外部系统的集成边界
- 系统设计·题5：人机协作流程怎么设计
- 系统设计·题6：高并发 Agent 服务的架构
- 系统设计·题7：Agent 的降级与兜底方案
- 系统设计·题8：从 0 到 1 落一个 Agent 项目怎么排期

九、大厂面经真题（按厂刷，最贴近实战）

字节（8 题）：题1 Agent 结合工具知识规划、题2 主从 Agent 通信与异常、题3 ReAct 实现与 tool_response 角色、题4 长任务上下文爆炸与 ReSum、题5 幻觉与 Structured Output、题6 Function Calling 与 RAG 区别、题7 大规模并发资源调度、题8 记忆系统与速度精度权衡

腾讯（8 题）：题9 多 Agent 系统设计、题10 跨境汇款超时与资金安全、题11 RAG 不停服更新、题12 千轮上下文与摘要、题13 分层而非单 Agent、题14 ReAct 落地、题15 RAG 全链路 chunk/Embedding/ReRank、题16 端到端成功率与监控

阿里（7 题）：题17 核心模块联动、题18 短期长期记忆、题19 推理断层与目标偏离、题20 Function Calling 兜底、题21 多 Agent 组织模式、题22 一万篇文档 RAG、题23 LoRA/DeepSpeed 显存估算

美团（7 题）：题24 智能客服多轮、题25 会话状态与模糊意图、题26 工具异常容错、题27 多知识库冲突、题28 并发打 API 卡死、题29 LangGraph 是否过度设计、题30 编排模型工具观测四层与回滚

> 刷题建议：大厂原题最贴近真实考察，建议先刷第九章摸底，再按上面九根轴回补薄弱章节。每题正文都给了考察点、开口怎么答、回答框架、面试官追问和避坑，照着练开口比背答案有用。